

Contents

RVSPOC 2026 复现验证材料·总览与裁定卷 1

 一、本包结构 1

 五、当前状态速览 1

 六、阅读时请注意的三点 2

 七、版本与修订 2

 八、终裁记录（2026-09-29） 3

第一部分 复现验证与评审阶段性报告 5

第二部分 待评判组裁定事项 21

第三部分 RVV 覆盖率事实表 31

第四部分 验证平台环境定版快照 36

第五部分 各题 Neon 算子分母表 39

RVSPOC 2026 复现验证材料·总览与裁定卷

日期：2026-09-29 编制：组委会验证侧 状态：**裁定已全部落地，结论定稿**

本包共 3 卷 PDF，覆盖五道赛题、22 个冻结提交中的 21 个有效提交

收件范围：评判组与各平台赞助方

版本 v2.3 发布日期 2026-09-29 本包 3 卷共用同一版本号

一句话说明：本包是组委会对五道赛题全部有效提交的复现验证记录。数据与验证过程载于本包各卷；**v2.2 及以前悬置的 14 项规则口径已全部裁定落地**，各题冠军（或无人达标）结论见第八章「终裁记录」。本版为公示版，与仓库公示目录 Results/Verifications/ 及官网公告同一口径。

一、本包结构

本次交付合并为 3 卷，取代此前分散的 17 份文件。

卷一 总览与裁定卷（即本卷） 第一部分 复现验证与评审阶段性报告 —— 五题全貌、方法学、数据可信度分层 第二部分 待评判组裁定事项 —— **需要评判组行动的 14 项** 第三部分 RVV 覆盖率事实表 第四部分 验证平台环境定版快照 第五部分 各题 Neon 算子分母表

卷二 A210 卷 —— P2601 KleidiCV、S2602 LiteRT 的详细验证记录（含 VLEN 三档、六提交精度与时延原始记录、双板差异、S2602 精度参考）

卷三 蓝芯 LX5000 卷 —— S2603 Redis、S2604 Memcached、S2605 RocksDB 的详细验证记录（含编译与正确性、性能与边界带复核、make check 失败归因、热点函数表、组委会标量基线）

只读本卷第一、二部分即可掌握全部结论与待决事项。卷二、卷三供查证结论时翻阅。

五、当前状态速览

五题的数据完备度

赛题	平台	有效提交	数据状态
P2601 KleidiCV	A210 b1	4	正确性与 benchmark 完整

S2602 LiteRT	A210 b2	6	精度、时延、算子级精度均已完整
S2603 Redis	LX5000	4	完整（#2/#6 编译失败未进入后续环节）
S2604 Memcached	LX5000	3	完整
S2605 RocksDB	LX5000	4	完整；60 分钟压测已完成 1 轮

21 个有效提交中，结论已全部落定（v2.3 终裁记录见第八章）：S2602 冠军 #6、P2601 冠军 #4（经与赞助方共同评审，综合比评）、S2605 冠军 #3（经与赞助方共同评审，按最接近达标、综合质量最优评定）、S2603 与 S2604 无人达标。

v2.2 及以前标注的「悬而未决」状态均已闭合，各提交的最终结论以第八章终裁记录与仓库公示目录为准。

六、阅读时请注意的三点

其一，本包如实记录了组委会自身的操作问题，共 12 类。其中 7 次若未察觉将造成误判，包括两个提交曾被错误记为「构建失败」——而实测表明其一性能为全场最强、其一精度为满分。之所以完整披露，是因为不交代这些，数据的可信度便无从判断。详见《正 1》第三章。

其二，结论的证据强度已分层标注。经隔离实验证实者、经多方交叉对照者、经逐行代码核实者、以及单点观察未作对照者，在各报告中分别注明，未混为一谈。

其三，所引选手代码均为指向冻结 SHA 的永久链接，即评审基准的那一版，不随分支后续变动；链接可匿名访问，已逐条核验可达。

七、版本与修订

本包为 v1.0，17 份文件共用同一版本号，发布日期 2026-09-22。每份 PDF 的抬头均标注版本号与发布日期，可据以核对是否为同一批次。

日后若有修订，将整包升版重发（v1.1、v1.2……），并在此处列出变更内容，不单独替换个别文件——避免收件人手中出现新旧混杂、彼此矛盾的材料。

版本	日期	变更
v1.0	2026-09-21	首次发布（17 份分散文件）
v2.0	2026-09-22	结构调整 + 内容更新，详见下方
v2.1	2026-09-23	新增 P2601 的 I420/YV12 布局专项证据
v2.2	2026-09-24	新增 S2602 上游行为比对（自查）；P2601 补入评委倾向 #4 的意见
v2.3	2026-09-29	14 项裁定全部落地，新增第八章终裁记录；各题冠军定稿；本版为公示版

v2.3 相对 v2.2 的变更

v2.2 及以前悬置的 14 项规则口径已全部裁定，各题结论定稿（详见新增的第八章「终裁记录」）：S2602 冠军 #6、P2601 冠军 #4、S2605 冠军 #3、S2603 与 S2604 无人达标。第二部分 14 项待裁定事项各节保留原陈述（数据与口径后果不变），其终裁结果统一载于第八章，不在各节内逐条改写——以保留裁定过程的全貌。

另：S2602 板上原始数据（精度/时延/构建 TSV、算子级差分张量原件、过程日志、生成脚本）已全部入库仓库公示目录 Results/Verifications/S2602/raw/，v2.2 中「待板上补入」的标注随之闭环。

v2.1 相对 v2.0 的变更

新增一项此前不在组委会验证范围内的证据：**P2601 的 I420/YV12 布局专项**。该证据由评委于 09-22 独立测试取得，组委会于 09-23 以自有归档源码独立复现，失败计数与评委记录逐项吻合（#3 两组解码分别失败 96 与 80 例，#4 两组解码全过；编码布局一项两家同样失败 80 例）。

该项同时暴露组委会验证方法的一处盲区：**赛题指定的自带测试套件全部通过，不等于移植实现与上游原版行为一致**——组委会原定方案未设置「与上游行为比对」环节。此盲区已在本卷第一部分 2.1 节如实记录。

该证据为 P2601 的名次提供了此前不具备的比评依据，相关说明已补入第二部分 2.5。

v2.2 相对 v2.1 的变更

按 I420 事件暴露的方法盲区，对 S2602 补做上游行为自查：以六个提交各自 fork 点的上游 LiteRT 原版为参考，与其向量档输出张量比对。**结论：S2602 不存在「底座漂移」问题**——24 组中 5 组超阈值者与此前算子级差分所提提交、所涉模型、数值完全相同，即三家有缺陷者（#7/#2/#3）的问题均在向量化本身，标量底座与原版行为一致。#7 与 #2 的缺陷证据链由此闭合为三条独立方法。详见《A210 卷》。

v2.0 相对 v1.0 的变更

结构方面：17 份文件合并为 3 卷，按「总览与裁定 / A210 / LX5000」归卷，便于在即时通讯中阅读；原两份平台报告（A210-复现验证报告、LX5000-复现验证报告）不再单独成文——它们是第一部分按平台逐字切分的产物，全员收全套后即为重复内容。

内容方面：

1. **S2602 算子级精度已全部完成**（v1.0 时为六个提交中的三个）。新增发现：#3 的 MobileNetV2 INT8 存在最大 3 LSB 的临界超出，其分布为 1001 个元素中 422 个完全相同、516 个差 1 LSB、61 个差 2 LSB、仅 2 个差 3 LSB，性质与 #2 的功能性失效（最大 159/255 LSB）截然不同。详见第一部分第 4.4 节。
2. **新增第 14 项待裁定**（第二部分 3.7）：S2602 算子级 1 LSB 判据是否按字面逐元素执行。该裁定影响 #3 在该题的名次。待裁定事项由 13 项增至 14 项。
3. **悬而未决的提交由 2 个增至 3 个**：新增 S2602 #3。
4. 全部文件加注版本号与发布日期；文内交叉引用统一改为本包卷次与部次。

本包中标注《…》的均为本次交付的 PDF；以等宽体标注的 .md/.sh/.tsv 为组委会内部工作文件或板上原始数据，不随本次交付，如需查阅可另行索取。S2602 的板上原始数据已入库仓库公示目录（见 v2.3 变更说明）。

八、终裁记录（2026-09-29）

本章为 v2.3 新增。第二部分所列 14 项待裁定事项已全部裁定落地，本章记载各事项的终裁结论；各节原陈述（事实与口径后果）不变，以保留裁定过程的全貌。本章结论与仓库公示目录 Results/Verifications/ 及官网公告为同一口径。

一、各题终裁结论

赛题	冠军	依据
S2602 LiteRT	#6	唯一同时通过精度与时延两项门槛者（平均 97.0 ms $\leq$ 110 ms）；「多数达标」与「平均 $\leq$ 110ms」两种整体口径结论一致，无换算法换赢家的空间。#7 时延最快（55.8 ms）但三个 FP32 模型输出与 x86 参考不符，在「测试模型缺一不可」处出局
P2601 KleidiCV	#4	#3 与 #4 在指定套件上成绩相同（4496 用例零失败）；经与赞助方共同评审，以 I420/YV12 布局专项中原版兼容性表现、代码组织与工程交付质量综合比评，选定 #4（依据《验收备忘录》§ 5.3，启用理由即 #3 存在 96/80 个解码布局不一致用例而 #4 全部一致）
S2603 Redis	无	最好成绩 +19.1%（#4，pipeline=64），距 +30% 门槛 10.9 个百分点；原因为题目负载特性（非流水线场景瓶颈在网络 I/O），非实现质量差异
S2604 Memcached	无	最好成绩 +3.6%；性能与覆盖率两道门槛均不可达（Top 20 热点中自有代码仅 40%）
S2605 RocksDB	#3	按门槛口径（必测三项全部 $\geq$ +30%）无提交达标；经与赞助方共同评审，按最接近达标、综合质量最优评定 #3 为冠军——其为唯一两项必测超门槛者（readrandom +29.9%~+32.6%、seekrandom +31.7%），且唯一修复上游平台缺陷者（rdcycle $\rightarrow$ rdttime）

二、14 项裁定逐项终裁 决定存亡的六项（2.1–2.6）

序	事项	终裁
2.1	S2605 的 +30% 门槛口径	严格口径（三项全部 $\geq$ +30%）维持 ，无提交达标；冠军评定不走门槛豁免，由赞助方共同评审按「最接近达标、综合质量最优」另行评定（见上表）。启用该评定方式的理由：#3 两项超门槛且唯一修复上游平台缺陷，其余提交与门槛差距显著（#4 必测三项全负、#2 一项为负、#1 无法运行），不构成评定空间
2.2	S2603/S2604 的 RVV 覆盖率分母	两题均因性能门槛未达而无冠军；覆盖率分母口径不再影响结论（S2604 按任一口径均无法达到 70%，已按字面口径记录，见各题 README）
2.3	S2605 #1 的 Zbc 崩溃定性	判未达标 。赛题详情页指定验证平台为 LX5000（ISA 不含 zbc），DB::Open 即 SIGILL 且无运行时检测；选手本地 K3（含 zbc）验证不暴露此问题属平台差异，如实记录，不改变判定
2.4	S2603 #2/#6 的 LTO 构建失败处置	以条件组（-fno-lto）数据记录 ：#2 全场景 $\leq$ +0.1%、#6 最好 +16.5%，均未达门槛；#6 开启 AOF 后的断言崩溃已逐行核实，记录在案。两提交均未达标，原构建口径与条件组口径结论一致
2.5	P2601 #1 的并行库路线如何计分	计入评审、以其真实证据强度参与比较 。其 RVV 代码三档全过、无质量问题；但指定测试套件技术上无法施加（自写 8 项冒烟 vs 同侪 4496 用例）、benchmark 9 条 vs 同侪 372–381 条，证据强度弱于同题通过者，未获评冠军。「已验证」的含义差异在评审记录中如实标注
2.6	「默认构建启用」对默认关闭者的认定	涉及的 S2603 #6、S2604 #2 均以条件组数据参与比较且未达门槛；本项不再影响任何结论，按字面口径留档

规则口径与流程八项（2.7、3.1–3.7）

序	事项	终裁
2.7	S2605 #1 的 crc32c 是否记为覆盖	留档。门槛为 3/3 而该提交另有一项完全缺失，本项不改变其未达标结论
3.1	AI 辅助占比 90% 的认定	认定满足申报要求。 规则要求「说明使用方式及占比」、未设上限；涉事三提交均如实申报且说明决策与复核由作者完成。不影响任何提交的验证结论
3.2	公开通报草稿的发送审批	讨论期通报 不再发出 ——公示期 PR 回复已覆盖其全部功能且结论已定，再发「请选手解释」反而混乱。六条草稿归档
3.3	S2602 算子级精度的对比基准	采用同提交标量档 vs 向量档差分（双档差分），判据为赛题条款载明的 $FP32 \leq 1e-5$ 、 $INT8 \leq 1 \text{ LSB}$ ；该方法不依赖外部参考值，选手可自行复现，已随公示材料公开
3.4	RocketMQ 压测的执行口径	60 分钟压测 1 轮，数据完整可用（无 OOM、无损坏、不丢消息）；P99 因官方工具不输出而如实声明不可得。3 轮中位数不再补做
3.5	评审定名与 S2602 #7 队名	S2602 #7 队名经报名表核实为「用力的告别是眷戀べ O」；公示材料中各提交以 PR 号标识，队伍名以报名表核对记录为准
3.6	S2604 #4 的说明文件口径	认定满足（实质重于形式），该 PR 已因超时判无效，仅留档
3.7	S2602 算子级 1 LSB 判据	按字面逐元素执行。 #3 的 MobileNetV2 INT8 最大 3 LSB（1001 元素中 2 个）计为该算子级判据不通过，如实记录；其模型级精度 6/6 不受影响。该裁定不影响 #6 的冠军地位，也不改变 #3 未达时延门槛的结论

三、终裁后无遗留

上述裁定落地后，v2.2 中悬置的三个提交（P2601 #1、S2605 #3、S2602 #3）均已闭合：#1 以其证据强度参与比较未获评冠军；S2605 #3 获评冠军；S2602 #3 算子级判据如实记录、模型级结论不变、时延未达门槛的总体结论不变。**五题 21 个有效提交的验证与评定全部完结，无待办事项。**

第一部分 复现验证与评审阶段性报告

日期：2026-09-22 编制：组委会验证侧 状态：阶段性，非最终结论

验证平台：蓝芯 LX5000（S2603/S2604/S2605）、A210 双板 b1/b2（P2601/S2602）

对比基线：组委会自测标量基线（《LX5000 卷》第五部分）、x86 精度参考（《A210 卷》第四部分）

代码引用：均为指向冻结 SHA 的永久链接，即评审基准的那一版，不随分支后续变动

版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

结论先行：22 个冻结提交中 21 个有效。板上自动化验证已完成主体：S2604 一题的编译、正确性与性能数据完整；S2603 的性能数据于 09-20 补齐后亦完整，仅 #2/#6 因编译失败未进入后续环节；**S2602 六个提交的精度与时延数据亦于同日补齐，至此完整**；S2605 主体完整，但 #1 因崩溃无法产出正确性与性能数据；P2601 正确性完整。**尚有两项赛题明文要求的验证未出结果**（S2602 算子级精度、S2605 RocketMQ 60 分钟压测）——**两项均已于 09-20 至 09-21 打通并开始取数：**RocketMQ 已完成 1 轮 60 分钟压测且全部判据通过（第 2.5 节），算子级精度已出首个提交的比对结果，六个提交已全部完成（第 4.4 节）；**代码审查阶段的事实表仅完成 14/21，逐项判定一项未做。**另有 14 项规则口径须评判组裁定，其中 6 项直接决定提交存亡；在裁定前，多题无法给出最终结论。备忘录原定 9/19-9/20 汇总合议，时间线已到期而工作未完。

一、验证范围与完成度

1.1 提交总数

22 个冻结提交，S2604 #4 因创建时间晚于冻结点已于 09-01 判无效，**有效提交 21 个**。全部提交的冻结 SHA 与 pr-archive/MANIFEST.tsv 逐一核验一致。

1.2 各环节完成度

下表的「已产出结论」指该环节对该提交已有可审计的落盘结果，**不等于该提交通过**；一个提交因崩溃或编译失败而无法测出数据，只要该结论本身已定性并留档，亦计入已产出。

环节	已产出结论	缺口
形式审查	21/21	无
可复现性（编译）	21/21	环节已全部执行。其中 S2603 #2/#6 编译失败，处置待裁定（第五章）
正确性	19/21	仅缺 S2603 #2/#6（编译未过，未进入本环节）。S2602 #4 已于 09-20 补齐
性能	20/21	仅缺 S2605 #1（DB::open 崩溃，无法运行）。S2603 #4、S2602 #4、P2601 #2 三项均已于 09-20 补齐
代码审查	事实 14/21，判定 0/21	最大缺口，见第四章

须特别说明 S2603 #4：该提交编译通过，仅正确性有 15 项失败（第 2.3 节），但板上性能结果中不存在其任何记录。核查性能流水线脚本后查明，这并非遗漏，而是脚本把受测对象限定为「**已通过正确性验证**」者，#4 因 15 项失败被排除。

问题在于该规则既未写入任何对外报告，**也未被一致执行**：S2603 #2 与 #6 连正确性都因编译失败而根本没跑，却以 CONDITIONAL 组测了全部八个场景；而正确性已跑、仅部分失败的 #4 反被排除。同一题内两种处理并存，且判据从未对评判组披露。

组委会已于 2026-09-20 按与 #3 完全相同的口径补测 #4，以消除该不一致，结果见第 2.3 节。

补测结果使该排除规则的后果显现：#4 八个场景全部优于基线，pipeline=64 达 +19.1%，是 **S2603 全题性能表现最强的提交**（#3 最高 +5.6%，#6 条件组最高 +16.5%）。即被该未披露规则挡在门外的，恰是本题性能最好的一个。虽然它仍未达 +30% 门槛、不改变本题「无人达标」的结论，但若门槛口径或排名口径日后有变，该数据的有无将直接影响结果。**是否采信一个正确性未全过的提交的性能数据，属口径问题，本报告不代为决定，但数据本身不应缺席。**

1.3 按剩余工作量估算

已完成约 70%，剩余约 10.5–13.5 人日：RVV 覆盖率逐项判定 3–5 人日（约 1250 条，人工为主）、§ 4.E 其余检查项 2–3 人日（抄袭比对的机械筛查已于 09-20 完成，见 4.3.1）、RocketMQ 压测 2.5 人日加 1.5 个板夜、S2602 算子级精度 2 人日加 1–2 个板夜、交叉核对与收尾 1 人日。

二、各题结论

2.1 P2601 KleidiCV (A210 / b1)

PR	队伍	完整 API 套件	VLEN 128/256/512	benchmark 测项数	结论
#3	瑞福扫	4496 用例，三档零失败	三档全过	372	通过

#4	260818	4496 用例，三档零失败	三档全过	372	通过
#1	Leaplab	不适用，见下	自写 8 项 smoke，三档全过	9	通过，证据强度弱于前两者
#2	RVV 智能	三档均 SIGSEGV	未通过	381 (09-20 补采)	不达标

2.1.1 #2 的两处实质问题

其一，RVV 档 `kleidicv-api-test` 在 `RemapF32/0.RandomNoPadding` 处收到 signal 11 崩溃，对应实现为 `kleidicv/src/transform/remap_f32_rvv.cpp`。证据五重：真机两次单独复现均以退出码 139 结束；内核日志记录 `kleidicv-api-te[60925]: unhandled signal 11 code 0x1`；QEMU 三档 VLEN 下崩溃点完全相同；`--gtest_filter=-Remap*` 排除该族后 4446 个用例全部通过；#4 在同板同工具链下同一测试三档各 4496 用例零失败。三档崩溃点相同表明该缺陷与向量长度无关，属固定的内存访问错误。

其二，标量档（`-march=rv64gc` 加 `KLEIDICV_ENABLE_RVV=OFF`）链接失败，缺 `kleidicv_count_nonzeros_u8`、`kleidicv::neon::min_max_loc`、`kleidicv::neon::min_max` 等符号。机制为：关闭 RVV 后这些 API 在 RISC-V 上既无 Neon 实现（该目标在 RISC-V 平台不构建）、又无 RVV 实现、亦无标量兜底。对应 § 4.E 第 2 项「标量回退实现存在，无 RVV 环境可编译运行」，本项不达标。

该结论于 09-20 获得**独立佐证**：补采 benchmark 数据时，标量档的 `kleidicv-benchmark`（与 `kleidicv-api-test` 完全无关的另一个构建目标）以**同一组缺失符号**链接失败。两个互不相干的消费方在同一处失败，表明缺陷在 `libkleidicv.a` 本身，而非某个测试目标的构建方式所致。

2.1.2 #1 的路线问题

该提交不修改上游 Arm 源码，RVV 实现独立置于 `riscv/library/`，其 README 原文为「Upstream Arm sources are not modified」。由此产生三个连带事实：

- § 4.C 的正确性判据是「KleidiCV 自带测试套件（gtest，对照参考实现）通过」，而该套件在技术上无法施加于 #1——它接不入上游树。组委会只能运行其自写的 8 项 smoke 测试（每项输出两行），而 #3、#4 各运行了 4496 个用例。「**已验证**」的含义在三者之间并不等价。
- § 4.D 考察 benchmark「数据完整性」，其产出 9 条测项，而 #2 为 381 条、#3 与 #4 各 372 条。即三家同侪均在 370–380 条量级，#1 低两个数量级，并非个别比较的结果。
- 其算子是否计入 130 条 API 分母，属规则解释。

该提交的 RVV 代码本身在三档 VLEN 下全部通过，无质量问题。争点在路线是否符合「移植 KleidiCV」的题意。

I420/YV12 布局专项（2026-09-23 新增，证据由评委独立提供并经组委会复现）

本项不在组委会原定验证范围内。组委会对 P2601 的正确性判据取自赛题详情页所指的 KleidiCV 自带 gtest 套件，#3 与 #4 在该套件下各 4496 用例、三档 VLEN 全部通过、零失败。但**该套件验证的是与参考实现的数值一致性，不验证输出的内存布局是否与上游 26.03 一致**。评委于 2026-09-22 就此另行测试（记录编号 P2601-I420-20260922），发现了该套件覆盖不到的问题。

组委会已独立复现，计数逐项吻合。复现所用源码取自组委会自有的 pr-archive 归档（经整树逐文件比对，与评委所附冻结源码零差异；冻结 SHA 亦与本报告记录一致：#3 为 289ac3fb、#4 为 c59a7eb3）；测试驱动沿用评委的 `i420_reference_test.cpp`，因其参考函数逐字取自 26.03 原版

test/api/test_yuv420p_to_rgb.cpp 与 rgb_to_yuv420_neon.h，是判定「是否与原版布局一致」的必要基准。评委归档的 39 个文件 SHA-256 校验全部通过。

测试组	#3 RVV	#3 标量	#4 RVV	#4 标量
原始参数解码（160 组）	64 / 96	64 / 96	160 / 0	160 / 0
偶数尺寸解码（480 组）	400 / 80	400 / 80	480 / 0	480 / 0
偶数尺寸编码布局（480 组）	400 / 80	400 / 80	400 / 80	400 / 80

单元格为「通过 / 失败」。上表为组委会复现所得，与评委记录的 summary.json 逐项相同，四档测试进程退出码均为 1。

三点须一并呈现，不可只取其一：

其一，该缺陷与向量化无关，属标量实现的地址计算问题。标量档以 KLEIDICV_ENABLE_RVV=OFF、-march=rv64gc 构建（评委已反汇编确认无 RVV 指令、ELF 属性未声明 V 扩展），其结果与 RVV 档完全相同——组委会复现时进一步发现，两档输出日志的 SHA-256 逐字节一致（#3 两档同为 949f9af1...，#4 两档同为 ca0e871d...）。源码分析与之互证：输入宽 2、高 6、stride 6 时，V 三行字节起点，原版 26.03 为 43 / 48 / 49，而 #3 的编码与解码均为 43 / 44 / 49。

其二，编码布局一项两家同样失败。#3 与 #4 在该组均为 400/80。差异在解码——#4 两组解码全过，#3 分别失败 96 例与 80 例。评委结论中亦明确「双方编码都有缺陷」。

其三，评委已自行标注该测试的局限，组委会认为应当如实转述：编码参考系「从原始源码规则构造的独立参考」，未在 A210 上执行 Arm Neon 原版二进制，亦未覆盖全部颜色与编码精度；失败集中于 height=6/10 且 padding=4 的参数组合，属可重复的布局问题，非接近阈值的舍入差异。

对组委会自身方法的说明：本项暴露了组委会验证方法的一处盲区——自带测试套件全部通过，不等于移植实现与原版行为一致。该盲区同样可能存在于其他移植类赛题的验证中。组委会未在原定方案中设置「与上游行为比对」这一环节，此处如实记录。

原始材料与组委会复现产物存于 evidence/P2601-I420/。

2.2 S2602 LiteRT (A210 / b2)

精度判据：ImageNetV2 matched-frequency 10,000 张，FP32 Top-1 差异 ≤0.1%、INT8 ≤1%（对 x86 参考）。时延门槛 110 ms。已决口径：每模型取较优线程数；先严后宽。

PR	精度（6 组）	时延达标数	六模型平均时延	备注
#6	6/6	4/6	97.0 ms	唯一同时通过精度与时延口径
#3	6/6	3/6	133.7 ms	三个 INT8 达标（50/51/63 ms），三个 FP32 未达
#4	6/6	1/6	171.3 ms	09-20 以 gcc-13 重建后补齐。三个 FP32 与 x86 参考完全一致；仅 EfficientNet-Lite0 INT8 时延达标（73.9 ms）
#1	6/6	1/6	298.9 ms	EfficientNet-Lite0 INT8 为 3240 ms，较 #6 同模型慢 22 倍
#7	3/6	6/6	55.8 ms	三个 FP32 输出为随机噪声，其时延不可采信
#2	4/6	0/6	177.8 ms	两个 MobileNet INT8 精度归零。已排除构建产物与编译器两种可能，定性为代码缺陷，见 2.2.3

六个提交的数据现已完整。四个提交精度 6/6 (#6、#3、#4、#1)；按已决的「先严后宽」，无提交在逐模型口径下六项时延全达标（#7 虽 6/6 但精度出局），故转整体口径，而「多数达标」与「平均 ≤ 110 ms」两种算法结论一致：仅 #6 通过（4/6 达标、平均 97.0 ms），#3（3/6、133.7 ms）、#4（1/6、171.3 ms）、#1（1/6、298.9 ms）、#2（0/6、177.8 ms）均不过。

2.2.1 #7 的 FP32 路径失效

三个 FP32 模型 Top-1 分别为 0.13%、0.10%、0.11%，等同 1000 类随机猜测；三个 INT8 全部达标。独立三重对照（各 200 张，同一 harness、同一批张量）：#7 的库跑 MobileNetV2 FP32 为 0/200；#7 的库跑同模型 INT8 为 145/200；#6 的库跑同一 FP32 模型为 147/200。harness 与数据因素排除。该构建使用选手自己声明的 gcc-13.3.0，编译器因素亦排除。

其三个 FP32 的时延为全场最快（67.3 / 92.6 / 106.7 ms，分别较 #6 快 25%、20%、40%），但输出为噪声。速度与正确性须合并判读：在 FP32 路径不产生正确结果的前提下，这三项时延不构成有效的性能证据。三个 INT8 的时延对应正确输出，可采信，余量均在 77% 以上。

2.2.2 时延的线程数效应

线程数对时延的影响方向随模型相反。以 #6 为例，MobileNetV2 FP32 单线程 89.7 ms、四线程 219.2 ms（慢 2.4 倍）；MobileNetV2 INT8 单线程 157.4 ms、四线程 62.3 ms（快 2.5 倍）。各提交的最优线程配置亦不相同：#6 的 FP32 以单线程为优，#2 与 #1 则四线程全面更优。此为「每模型取较优线程数」口径的实据。

2.2.3 #2 的精度失败已确证，成因已于 09-21 判定为代码缺陷

该提交在 PR 描述中声明编译器为 GCC 15.2.0。实测其代码使用 __RISCV_VXRM_RDN（GCC 14 才引入的定点舍入模式常量），故 gcc-13 构建失败：

```
error: '__RISCV_VXRM_RDN' was not declared in this scope
  rvv_quantization_utils.h:37,46,55; rvv_tensor_utils.cc:223,246
```

第一步，排除组委会的构建产物问题（其库曾在多次中断的构建中增量产出）：以全新构建树、gcc-14 重做，结果如下（每项 500 张，同一 harness、同一批张量、同一模型文件）：

实验	MobileNetV2 INT8	MobileNetV1 INT8	EfficientNet-Lite0 INT8
#2 的库，gcc-14 全新构建	0.0%	1.4%	73.6%
#6 的库，同批图像（对照）	68.4%	69.8%	—

失败在全新构建树下稳定复现，并非组委会的构建产物问题；且失败范围精确限于 MobileNet 系的 INT8 路径——同一次构建的 EfficientNet-Lite0 INT8 输出正常（73.6%），三个 FP32 模型在万张全量测试中亦与 x86 参考完全一致。

第二步，排除编译器代码生成差异。此前该项停在「无法判定」，因为选手声明的 GCC 15 在验证平台（Debian 13 trixie）的软件源中不可得，无法排除失败系 gcc-14 与 gcc-15 的代码生成差异所致。09-21 组委会取 Debian sid 的 gcc-15 (15.3.0-3)，以 dpkg -x 解包至独立前缀而非系统安装——直接安装会将 libgcc-s1、libstdc++6 升级至 gcc-16-base 16.2.0，属全系统运行时库升级，会影响已编好的全部 S2602 产物；解包方式下系统库未受任何影响（核验 libstdc++6 仍为 14.2.0-19）。以该 gcc-15 重建 #2 后复测：

模型	gcc-14	gcc-15.3.0	是否复现
----	--------	------------	------

MobileNetV2 INT8	0.0%	0.0%	完全一致
MobileNetV1 INT8	1.4%	1.4%	完全一致
EfficientNet-Lite0 INT8 (对照)	73.6%	73.6%	完全一致

三个模型在两种编译器下的结果一字不差，包括作为对照的 EfficientNet-Lite0——它在两种编译器下都正常，说明差异不在编译器而在模型路径。

结论：该失败在选手声明的编译器大版本下原样复现，与工具链无关，属选手代码缺陷。两个 MobileNet 均大量使用 depthwise separable 卷积，其量化路径为首要嫌疑。

须声明的限度：选手声明 15.2.0，组委会实测 15.3.0，主版本相同、次版本不同。严格而言不能绝对排除 15.2 与 15.3 之间的差异，但同一大版本内出现「Top-1 由 0% 变为正常」这一量级的代码生成差异，可能性极低。本结论按**高度可信但非绝对**的口径给出。

该项原被列为 § 4.B 可复现性事项（选手指定了验证平台上不可得的工具链）；经上述验证，其**可复现性障碍已解除，问题定性转为代码缺陷**。

2.2.4 S2602 的代际差异

三家提交使用的 RVV 定点 intrinsics 分属两代 API：#7 与 #4 使用 3 参数旧式，仅 gcc-13 可编译；#2 使用 4 参数新式（含 `__RISCV_VXRM_RDN`），需 gcc-14 及以上。组委会初期统一使用 gcc-14，致 #7 与 #4 构建失败，已更正（详见第三章）。

两次更正对本题结论的影响须如实说明：#7 更正后成为时延最快的提交（6/6 达标、平均 55.8 ms），但其三个 FP32 输出为噪声，精度仅 3/6，在 § 4.C 「三模型缺一不可」即出局；#4 更正后**精度 6/6 全部达标**——其三个 FP32 的正确数（5,833/5,884/6,167）与 x86 参考一位不差，三个 INT8 的偏差为 +6/+10/+8（即 +0.06%/+0.10%/+0.08%），恰落在已查明的「numpy 银行家舍入 vs C++ `std::round`」系统性偏差区间内——但其时延仅 1/6 达标、平均 171.3 ms，在精度达标的四家中排第三。

两次更正均未改变本题的领先者——#6 仍是唯一同时通过精度与时延口径者。但更正对这两个提交自身影响极大：**若不更正，二者都会以「构建失败」被整个淘汰**，而实测表明 #7 的性能本是全场最强、#4 的精度本是满分。这两例说明，以统一配置替代选手文档所声明的工具链，足以使一个合格提交在最初一关即出局。

2.3 S2603 Redis (LX5000)

#3 正确性通过。#4 出现 15 项测试失败，机制已定位为两处。

其一，`src/monotonic.c#L187` 新增的 RISC-V 分支在无法确定时钟频率时无条件写 `stderr`：

```
fprintf(stderr, "monotonic: riscv, unable to determine clock rate\n");
```

而 Redis 测试经 `Tcl exec` 调用子进程，子进程写 `stderr` 即判错（即便退出码为 0），由此 9 项 benchmark 集成测试失败。上游同文件的 x86（第 87 行）与 aarch64（第 130 行）分支亦有同样的打印，但二者在对应平台上能取得频率，故不触发。

其二，`src/memory_prefetch.c#L305` 与 `#L432` 处：

```
void dictPrefetchKeys(dict **dicts, void **keys, size_t nkeys) {
    #if defined(__riscv) && !defined(__riscv_zicbop)
        return;
    #endif
```

`__riscv_zicbop` 为编译器宏，仅当 `-march` 显式包含 `zicbop` 时定义，与 CPU 是否实现该扩展无关。该提交构建 `-march=rv64gcv`（不含 `zicbop`），故 Prefetch 功能被整体禁用；而验证平台实测 ISA 含 `_zicbop`。测试断言 `io_threaded_total_prefetch_entries` 在 2 至 15 之间，禁用后计数恒为 0，由此 4 项 Prefetch 测试及 2 项关联测试失败。

#2 与 #6 在默认构建（Redis 的 `-flto=auto`）下因 LTO 与向量扩展冲突而编译失败，仅追加 `-fno-lto` 即可编过，**该编译失败本身不涉及代码错误**，处置待裁定（第五章）。但须注意：#6 在加 `-fno-lto` 后的条件组测试中，开启 AOF 即触发服务端断言崩溃（Guru Meditation: You tried to retain an object allocated in the stack, object.c:637）。机制已逐行核实：其为 GET/SET 的键做零拷贝优化，在查询缓冲区内就地伪造 SDS 头，以 `initStaticStringObject` 将缓冲区字节借用为静态引用计数对象（[src/networking.c#L3509-L3518](https://src.networking.c#L3509-L3518)）；开启 AOF 后 `alsoPropagate` 对该对象调用 `incrRefCount`，而后者遇 `OBJ_STATIC_REFCOUNT` 即 panic。故 #6 即便获准以 `-fno-lto` 继续评审，仍须先过正确性关，必然命中该崩溃。完整因果链与代码原文见 本卷第二部分 第 2.4 节。

性能（八个场景，每场景 3 轮取中位数，对组委会标量基线）：

提交	组别	最好场景	全场景区间	达 +30%
#4	正式（09-20 补测）	pipeline=64 为 +19.1%	+0.5% 至 +19.1%，八场景全部为正	否
#6	条件（ <code>-fno-lto</code> ）	pipeline=64 为 +16.5%	无 AOF 的七场景；AOF 场景崩溃	否
#3	正式	base 为 +5.6%	-1.5% 至 +5.6%	否
#2	条件（ <code>-fno-lto</code> ）	+0.1%	全场景不高于 +0.1%	否

条件组数据是否采信取决于第五章对 LTO 处置的裁定。**无提交达到 +30% 门槛。**

#4 的增益分布与其优化手段自治，值得评判组注意：其八个场景中，增益最高的两项恰为 `pipeline=64`（+19.1%）与 `pipeline=16`（+12.1%），而非流水线场景均在 +5% 以内。这与第 2.6 节的热点结论一致——非流水线场景的瓶颈在网络 I/O 与系统调用，向量化无从着力；开启流水线后系统调用开销被摊薄，协议解析在时间占比中上升，而 #4 正是四个提交中唯一向量化了 RESP 扫描者（`src/rvv_resp.c`，其 `processInputBuffer` 路径调用 `redisRespScanFindByteRvv`，已人工核实，见 本卷第三部分 第 4.2 节）。**即优化手段、热点分析与实测增益三者互相印证**，而非孤立的数字。

须一并说明：#4 是该题性能表现最强的提交，却因第 1.2 节所述的未披露排除规则而长期无任何性能数据。该数据于 09-20 补齐。

2.4 S2604 Memcached（LX5000）

#1、#2、#3 正确性全部通过。性能：全部场景对基线的差异在 -2.7% 至 +3.6% 之间，无人达标。

2.5 S2605 RocksDB（LX5000）

`db_test` 与 `make check` 均已执行。三个受测提交（#2、#3、#4）无任何一项失败归因于选手：全部失败归为两类，均经隔离实验定性。其一为 `range_locking_test` 的 14–16 项 SIGILL，根因是上游 `toku_time.h:155` 的内联汇编 `rdcycle`，而 Linux 自 6.6 起禁止用户态读取该寄存器（本板内核 6.18.3+），属上游固有缺陷；其二为 `options_settable_test` 的结构体 `padding` 计数偏差，隔离实验证实上游源码在同一工具链（gcc-14）下同样失败且偏差同为 24 字节，属编译器代码生成所致。

#3 在其说明文件中记载并修复了上述 `rdcycle` 问题 ([toku_time.h#L141-L147](#)，注释原文为「Use `rdtime`, not `rdcycle`: Linux >= 6.6 traps user-mode `rdcycle` (SIGILL) for security」)，实测其 `make check` 测试阶段 38,934 个用例全部通过，为受测提交中唯一修复上游平台缺陷者。

#1 因使用 Zbc 标量 `clmul` 指令而 `DB::Open` 即 SIGILL 崩溃 (gdb 定位至 `util/crc32c_riscv.cc:66`)，`db_test` 无法运行，`make check` 与 RocketMQ 压测对其均无参考意义。

性能：严格口径（必测三项全部 $\geq +30\%$ ）实测无提交通过。

60 分钟长稳压测结果（#4，2026-09-21，1 轮）

负载按赛题三要素并发施加：128 B $\times$ 16 线程混合收发、4 KiB $\times$ 8 线程大消息、消费者延迟 600 秒启动形成积压。存储 `storeType=defaultRocksDB`，单 Broker + NameServer。

指标	128 B $\times$ 16 线程	4 KiB $\times$ 8 线程
平均发送 TPS	15,585	9,478
平均 RT	1.025 ms	0.842 ms
最大 RT	794 ms	689 ms
发送失败	0	0

判据达成情况：

判据	结果
两个消费组 Consume Diff Total 均为 0（不丢消息）	通过（ <code>task5_small</code> 与 <code>task5_large</code> 均为 0）
Broker 日志无 OOM	通过（ <code>OutOfMemory</code> 、OOM 计数均为 0）
无数据损坏	通过（ <code>Corruption</code> 计数为 0）
无后台错误	通过（ <code>Background error</code> 计数为 0）
60 分钟持续运行	通过（生产 09:44:00–10:44:01 UTC，续 10 分钟排空积压）

30 秒一次的采样共 130 点，内存可用量在 243.1 GB 至 238.8 GB 之间平稳波动，无单调下降趋势；磁盘由 1.6 TB 降至 1.5 TB，`store` 最终占用 167 GB。

两处须声明的限度：

其一，**P99 未取得**。RocketMQ 官方 benchmark 工具只输出 Average RT 与 Max RT，不输出 P99（S2605 #3 已在其 `docs/COMPLIANCE.md` 公开承认该限制）。#4 文档亦特别提醒 P99 须由外部采样器从同轮请求延迟计算，**不可用 Max RT 代替**。本轮未部署该采样器，故 P99 如实声明为不可得，处置方式待评判组裁定（第 8.7 节）。

其二，**采样脚本对 NameServer 的探活失效**，130 个采样点均记为 `ns=down`。经核查，这是观测方式的缺陷而非真实中断：`bin/mqnamesrv` 是 shell 包装脚本，它以 `sh runserver.sh` 拉起 JVM 后自身退出，故组委会记录的 PID 早已不存在。实际证据表明 NameServer 全程正常——压测时段内其日志有 2,472 行活动记录，Broker 侧「连接 NameServer 失败」计数为 0，且消息全程正常流转。Broker 的探活（`broker=up`）130 点全部正常，不受此影响。

本轮为 1 轮。§ 4.D 的「每项 3 轮取中位数」是否适用于 60 分钟稳定性测试尚待裁定（第 8.7 节）；若裁定为适用，再补跑 2 轮即可，本轮数据不作废。

2.6 S2603 与 S2604 全员未达 +30% 的原因

热点采样（《LX5000 卷》第四部分，标量版在验证平台 perf 采样 Top 20）显示：S2604 的首位热点为 pthread_mutex_lock，占 14.26%，锁原语合计 5 项 25.4 个百分点，系统调用 3 项 19.7 个百分点，自有代码仅 8 项 17.8 个百分点；S2603 base 场景的 read/write/vDSO 合计 20.2 个百分点。瓶颈在网络 I/O 与锁竞争，向量化无法触及。按 Amdahl 定律，即便协议解析被完全向量化，+30% 在这两题上亦不可达。此为客观的题目特性，非选手实现质量问题。

该推断已获实测佐证，不再仅是理论论证。09-20 补齐 S2603 #4 的性能后，本题出现了一个恰好可用于检验该推断的样本：#4 是四个提交中唯一向量化 RESP 扫描者，其增益分布与上述热点结构完全吻合——

场景类型	#4 对基线	与热点结构的对应
pipeline=64	+19.1%	系统调用被流水线摊薄，协议解析占比上升，向量化得以发力
pipeline=16	+12.1%	同上，摊薄程度较低
六个非流水线场景	+0.5% 至 +4.4%	瓶颈仍在 read/write/vDSO，向量化收益被稀释

即：在向量化最能发挥作用的流水线场景下，由唯一命中协议解析热点的实现取得的最好成绩仍只有 +19.1%，距 +30% 尚有明显差距。这是该门槛在本题不可达的直接证据，强于单纯的 Amdahl 推演。S2604 无对应样本，其论证仍以热点结构为据（自有代码仅占 Top 20 的 8 项、17.8 个百分点）。

三、组委会侧的操作问题

共发现 12 类，其中 7 次若未察觉将造成误判（第 3、4、9、11、12 项直接使提交的数据缺失、被错误淘汰或被错误归因）。全部已更正并留档。

序	问题	后果与更正
1	S2604 #3 使用简化构建命令，遗漏选手文档中的 memcached-debug 等目标	曾误判 testapp 失败；按文档完整目标重建后通过
2	P2601 #3 的完整 API 套件被 KLEIDICV_BUILD_FULL_API_TESTS（默认 OFF）挡住	曾仅运行该套件在关闭状态下残留的 8 项冒烟用例（与第 2.1.2 节所述 #1 自写的 8 项 smoke 无关，两者恰好同为 8 项），而 #4 运行了 4496 个用例，证据强度不对等；已显式开启补测，结果为三档各 4496 用例零失败
3	S2602 #7 使用 gcc-14 构建，而其 PR 描述明载「13.3.0 即可，不需要 GCC 14」	曾记为 BUILD_FAIL；改用 gcc-13 后构建成功，且成为该题时延最快的提交
4	S2602 #4 同因（3 参数定点 intrinsics），组委会以默认的 gcc-14 构建	原 BUILD_FAIL 记录已被推翻 ：09-20 改用 gcc-13 重建成功（1323 秒）。补测结果为 精度 6/6 全部达标 （三个 FP32 与 x86 参考一位不差）、时延 1/6。即一个精度满分的提交，原本会以「构建失败」被淘汰
5	LTO 实验对 S2603 #6 漏传 BUILD_RVV=yes	该仓库的 RVV 对象仅在此开关下参与编译，原实验为无 RVV 的空验证；已带全套文档开关重做，结论不变
6	diff 基线误用首个 commit	提交仓库为上游完整 fork，55 个真实改动被算成 8022 个；改用 git merge-base origin/HEAD

7	make check-format 就地改写选手源码	RocksDB 的 format-diff.sh -c 自动向 9 个新增文件写入版权头；已 git checkout 全部还原，并核验 HEAD 仍为冻结 SHA fa3fd926
8	性能受测范围的排除规则未披露、且未一致执行	流水线把受测对象限定为「已通过正确性验证」者，S2603 #4 因 15 项失败被排除，但 #2/#6 连正确性都未跑却测了 CONDITIONAL 组；该判据从未写入任何对外报告。已于 09-20 按同口径补测 #4（第 1.2 节）
10	A210 两板均未安装裸 g++（b2 有 g++-13 与 g++-14，b1 只有 g++-14）	凡不继承父构建编译器设置的外部项目（CMake ExternalProject）都会失败，且报错为「找不到编译器」而非真实原因。今日两次绊倒：P2601 #2 的 benchmark 补采、S2602 kernel 单测所需的 flatbuffers-flatc。09-20 已在 b2 装上默认 gcc/g++（14.2.0），该类故障不再复现
9	P2601 #2 的 benchmark 被误记为「未采」	实为构建 all 目标时撞上 examples/C/kleideicv-c-example 链接失败，整次构建被记为 FAIL，benchmark 遂未运行——而同一日志显示 kleideicv-benchmark 本身「Built target」成功。09-20 改为只构建该目标后补采成功，产出 381 条测项（与 #3、#4 的各 372 条同量级）。原记录使该提交看似缺失 § 4.D 数据，实为组委会构建方式所致
11	验证平台 JDK 版本与 RocketMQ 5.5.0 不匹配	RocketMQ 5.5.0 依赖的 spotbugs-maven-plugin:4.2.2 在 JDK 21 下抛 UnsupportedOperationException: The Security Manager is deprecated，首次构建在 rocketmq-common 失败。该项目面向 JDK 8/11，而组委会装的是 JDK 21。跳过该静态分析插件后 18 个模块构建通过。若未查明，会被误记为 S2605 #4 的 JNI 集成失败
12	RocksDB JNI 构建漏开压缩支持	RocksDB CMake 的 WITH_LZ4/WITH_SNAPPY/WITH_ZSTD/WITH_ZLIB 默认均为 OFF，组委会首轮 JNI 构建未显式开启。而 RocketMQ 的 ConsumeQueue 存储要求 LZ4，RocksDB.open 遂抛 Compression type LZ4 is not linked with the binary，后续 this.db 为 null 触发 NPE，Broker 启动失败、生产者 16 万条发送失败。若未查明，会被误记为 S2605 #4 的 JNI 集成不可运行。带 -DWITH_LZ4=ON 等重建后，五个 RocksDB 存储全部 starts OK，Broker boot success

第 1 至 4 项与第 9 项的共同原因是组委会采用统一配置，而各提交的构建系统并不一致。第 9 项另暴露一处此前未察觉的细节：流水线一直在传 -DKLEIDICV_BUILD_EXAMPLES=OFF，但 P2601 #2 的顶层 CMakeLists 无任何 option() 声明、且无条件 add_subdirectory(examples/C)，该参数对其静默失效（CMake 缓存中记为 UNINITIALIZED）；而 #3、#4 将同一语句置于条件块内并各声明 4–5 个 option，同一参数对其有效。即同一条构建命令在三个提交上的行为并不相同。

第 8 项性质不同，是判定口径未落纸、仅隐含在脚本中。

此外，09-20 的核对中还避免了一次潜在污染：为绕开 S2602 #4 的依赖抓取失败，曾拟借用 #2 已下载的第三方依赖源码，比对后发现两者锁定的 abseil 并非同一 commit（#2 为 255c84da，#4 为 d38452e1），遂只借用了经逐字节比对确认一致的 8 项依赖，abseil 仍用 #4 自己抓取的那份。此例说明构建树不可跨提交共用，已记入操作纪律。

四、代码审查 (§ 4.E) 现状

4.1 RVV 覆盖率 (门槛项)

规模为 21 个有效提交、约 1250 条逐项判定, 其中 S2602 一题占 588 条。当前进度:

- **S2605**: 分母 3 个模块 (90% 门槛即 3/3 全覆盖), 事实表已完成。#3 与 #4 为 3/3, #2 为 1/3; #1 视 `crc32c` 一项是否记为覆盖而为 2/3 或 1/3——该模块存在 RVV 路径但在验证平台崩溃, 属第五章待裁定事项。**需注意: 门槛为 3/3, 故无论如何裁定, #1 与 #2 在本门槛项上均不达标**, 该裁定影响的是记录口径而非 #1 的存亡。
- **P2601**: 分母 130 个 C API 入口。已完成按算子类别的 intrinsic 分布统计, 逐 API 判定未做。
- **S2603/S2604**: 已完成向量化方向清单与三项已人工核实的热点命中, 判定口径未定。
- **S2602**: 分母已于 09-20 定稿为 98 项 (门槛 89/98), 逐项判定未启动。

4.2 覆盖率不以百分比呈现的原因

组委会曾尝试自动化判定 P2601 的逐 API 覆盖率, 同一份代码在两种严格度的判据下结果相差悬殊, 且各提交的落差幅度互不相同: #4 为 62.3% 对 21.5% (约 3 倍)、#3 为 65.4% 对 14.6% (约 4.5 倍)、#2 为 53.1% 对 37.7% (约 1.4 倍)。判据的松紧竟能改变三家的相对次序, 说明该方法不可靠。已实证的原因有三:

其一, 三家提交的代码组织粒度不同, 文件名到 API 的映射有损。#2 为每算子一个小文件 (约 50 个), #3 为每类别一个大文件 (9 个, 其 `transform` 类合计 565 处 intrinsic 中, `transform_rvv.cpp` 单文件即占 557 处), #4 居中 (18 个)。

其二, 「函数落在 `rvv` 命名空间」不等于「函数体真的向量化」。#4 的 `canny_rvv.cpp` 共 198 行, 确在第 16 行的 `namespace kleidicv::rvv` 内, 但全文含 0 个 `__riscv_v*` 调用, 为纯标量实现, 且整个文件被第 14 行的实验性开关守护:

```
#if KLEIDICV_EXPERIMENTAL_FEATURE_CANNY
namespace kleidicv::rvv {
```

其三, API 测试全部通过不能证明全部向量化。#3 与 #4 的 4496 个用例零失败, 但二者含真实 intrinsic 的文件分别仅 9 个与 18 个; 多数 API 落在 `rvv` 命名空间内的标量实现上, 而测试验证的是正确性。

因此 本卷第三部分 只给出可核验的事实 (按类别的 intrinsic 分布、已人工核实的命中), 逐项判定须人工逐条核实函数体。

4.3 其余检查项

标量回退存在性与条件编译隔离两项可随覆盖率扫描一并产出, 已部分获得 (如 P2601 #2 的标量回退缺失系构建时实测所得)。S2603 专项 (无多线程优化、状态机边界未变) 与代码规范两项中, S2603 专项已于 09-20 完成 (见 4.3.2)。

4.3.3 代码规范: 已于 09-20 执行, 但两种口径结论差别很大, 须一并呈现

仅 S2605 (RocksDB) 与 P2601 (KleidicV) 自带 `.clang-format`, S2604 (memcached) 与 S2603 (Redis) 的 C 代码无统一格式配置, 四题判据本就不齐, 故本项只覆盖前两题的 8 个提交。执行方式为 `clang-format --dry-run` (只报告不改写)——刻意不使用 RocksDB 自带的 `build_tools/format-diff.sh`, 因其会就地改写源码 (组委会已因此误改过 9 个文件并还原, 见第

三章第 7 项)。受检范围限于各提交相对上游 git merge-base 的新增与修改文件，未动过的上游代码不计入。

先做了对照实验以验证工具本身可信：对三个仓库各抽 40 个选手未修改的上游文件施加同一检查，偏离率均为 0%。即 clang-format 19 与这些项目既有的格式完全一致，后续测得的偏离不是工具版本差异造成的假象。

提交	受检文件	偏离文件	文件偏离率	受检行数	建议改动行	行偏离率
P2601 #1	151	123	81%	17,000	2,194	12.9%
P2601 #3	82	55	67%	23,241	2,936	12.6%
P2601 #2	54	44	81%	13,197	833	6.3%
P2601 #4	81	36	44%	15,437	585	3.8%
S2605 #2	12	5	42%	4,165	148	3.6%
S2605 #1	18	9	50%	6,857	51	0.7%
S2605 #3	44	16	36%	37,584	150	0.4%
S2605 #4	27	9	33%	34,319	60	0.2%

两种口径不可混用：按「文件是否偏离」计，八个提交的偏离率在 33% 至 81% 之间，看似普遍严重；但该指标是二值的——一处空格与整文件不合规同等计一次。按行计，实际幅度为 0.2% 至 12.9%，差出一至两个数量级。建议以行偏离率为准，文件偏离率仅作分布参考。

按行计的结论是：S2605 四个提交均在 3.6% 以内，其中 #3、#4 低于 0.5%，格式贴合项目规范；P2601 的 #1 与 #3 在 12.6%–12.9%，明显高于同题的 #2（6.3%）与 #4（3.8%）。本项无赛题明文的达标线，上述数字供评判组按其自定义权重使用，本报告不代为判定。

4.3.1 抄袭比对：机械筛查已完成，未发现抄袭信号

做法：对每个提交取其相对上游 git merge-base 的新增实质代码行（剔除空行、注释、#include、纯括号，只保留长度超过 15 字符者），再在同一赛题内两两求交集，以较小一方为分母计算重合比例。不可比整文件——各提交改的多是同一批上游文件，整文件比对会得出九成以上的虚高相似度（例如 S2603 四个提交的 deps/xxhash/xxhash.h 相似度为 1.0，实为未改动的上游代码）。

除两对外，全部组合的重合比例均在 11% 以下，与「使用相同 intrinsic 独立实现」相符。两处高值经查证均有正当解释，不构成抄袭：

组合	重合	查证结果
S2602 #1 (Leaplab) 与 #3 (算力土豆)	3613 行，占 #1 的 77.5%	其中 3421 行来自 test/rvv/third_party/stb_image.h——公有领域的第三方图像解码头文件，两队各自引入同一公开文件，路径本身即标注 third_party。扣除后实际重合 192 行，散落于 RVV GEMM 头文件，属相同 intrinsic 序列的正常相似
S2605 #3 (SNL) 与 #4 (260818)	323 行，占 #4 的 49.6%	全部落在 table_test.cc、sst_file_reader.cc 等上游 RocksDB 文件，内容为上游测试断言，与 RVV 无关。成因是两者 fork 点不同，介于两个 fork 点之间的上游提交被 merge-base 算作「新增」

本项的局限，须明确：该筛查为逐行精确匹配，只能发现原样复制；经改写变量名、调整结构或重排语句的抄袭无法检出。筛查范围亦限于同一赛题内的提交两两比对，**未覆盖跨赛题比对，也未比对公开的第三方 RVV 项目**——后者需外部检索，尚未进行。故本节结论应表述为「未发现原样复制」，而非「确认无抄袭」。

4.3.2 S2603 专项：无多线程优化、状态机边界未变

赛题对 S2603 另有两条专项约束。就新增代码逐行核查，结果如下。

其一，**无多线程优化——四个提交全部符合。**在各提交相对上游 merge-base 的新增行中检索 pthread_create/join/detach、std::thread、#pragma omp、线程池等关键字：#2、#3、#6 命中为零；#4 有三处命中，但均为**读取** Redis 上游已有的 server.io_threads_num，且对应的删除行显示上游同一函数本就读取该变量，属对既有分支的重构，未引入新的并行。

其二，**状态机边界——状态变量无一被改写，但 #6 有一项须评判组注意的做法。**检索 c->reqtype、PROTO_REQ_INLINE/MULTIBULK、c->multibulklen、c->bulklen、c->qb_pos 等状态量：四个提交的新增行中，**对这些状态量的赋值为零**，全部为读取后据以选择是否走向量化扫描，协议状态机的转移逻辑未被改动。

但 #6 虽未改状态变量，却在**查询缓冲区内部就地写入**：为 GET/SET 的键伪造一个 SDS 头（arg_data[-1] = SDS_TYPE_5 | ...，arg_data 即 c->querybuf + c->qb_pos），覆写的是缓冲区中已解析位置之前的字节，再将该段字节借用为静态引用计数对象。这不改变状态机的转移，但改变了「解析缓冲区只读」这一隐含约定，**并且正是其 AOF 断言崩溃的根因**（因果链见 本卷第二部分第 2.4 节）。该做法是否越过「状态机边界未变」的要求，属规则解释，本报告不代为判定。

4.4 S2602 算子级精度：原定路径不可用，已改走输出张量差分

原定路径为开启上游 TFLITE_KERNEL_TEST=ON，运行其 137 个 kernel 单测——这些单测本身就是「优化实现 vs reference_ops」的逐元素比对，基准在测试内部，且选手无法篡改阈值，是最贴合 § 4.C 的做法。

实测该路径在本版本 LiteRT 上不可用。组委会逐一定位出五处上游自身的缺陷：

序	缺陷	处置
1	LiteRT 独立仓库根下无 TF 单体仓库的 compiler/ 目录，而 kernels/CMakeLists.txt 引用 \${TFLITE_SOURCE_DIR}/../compiler/...	符号链接补位
2	所引 tools/versioning/op_version.cc 在本版本已移至 converter/tools/versioning/	符号链接补位
3	XLA 的 xla/tsl/platform/default/logging.cc 已上移一级	符号链接补位
4	xnnpack_delegate_provider.cc 被无条件列入测试框架源，而该文件全文无 任何 守卫宏，XNNPACK 关闭时仍调用其 API	改以 TFLITE_ENABLE_XNNPACK=ON 构建
5	测试框架的 quantization_utils.cc 传递依赖 TensorFlow core 的 protobuf 生成头 (types.pb.h)，而 CMake 构建路径从不运行 protoc 生成它们	无法绕过

前四处已处置，configure 通过且 **169 个 kernel 测试目标成功注册**；第五处为结构性障碍——补齐它等于在 CMake 路径上重造 TensorFlow 的一部分构建系统（那本是 Bazel 的职责），代价与方法学风险均不相称，故终止该路径。

这五处与选手全部无关：六个提交对其中任何一处均无改动（xnnpack_delegate_provider.cc 的 md5 六者完全相同）。由此可得一项对评审有直接影响的事实—— § 4.C 所要求的算子级精度，其上游自带测试设施在选手所用的这个 LiteRT 版本上是开箱即坏的。这既解释了为何无选手跑通该套件，也意味着不宜以「选手未做算子级验证」为由扣分。

改用的路径：同一提交分别以 -march=rv64gc（标量）与 -march=rv64gcv（向量）构建，对同一输入 dump 输出张量，逐字节比对；标量档即「同提交的参考实现」。该口径取自选手 #4 自带的 scripts/compare-model-differential.py——它是唯一把 § 4.C 的 $1e-5$ 与 1 LSB 直接写进断言的工具，比对范围覆盖 VLEN 128/256/512 三档。采用选手自己的方法学，可最大限度减少口径争议。组委会为此在验证 harness 中增加了 dump 模式（输出张量按原始字节落盘，不做任何归一化或重排，以使比对无损且可被选手独立复现）。

首份实测数据（#6，2026-09-21）：标量档（rv64gc）与向量档（rv64gcv）对同一输入的输出张量比对结果如下。标量档即「同提交的参考实现」，判据取 § 4.C 的 $FP32 \leq 1e-5$ 、 $INT8 \leq 1$ LSB。

模型	类型	输出元素数	数值不同的元素	最大绝对差	判定
MobileNetV2	FP32	1001	1001	2.38e-07	通过 ($\leq 1e-5$)
MobileNetV2	INT8	1001	0	0	通过 (逐位相同)
MobileNetV1	FP32	1001	848	1.16e-11	通过 ($\leq 1e-5$)
MobileNetV1	INT8	1001	0	0	通过 (逐位相同)

即 #6 的向量化实现未改变数值语义：两个 INT8 模型的输出与标量档逐位相同，两个 FP32 模型的最大偏差比阈值小两个数量级以上，属浮点重结合的正常误差。

方法学须注明一处：比对必须按数据类型解析后再算差值。组委会初次以逐字节比较，FP32 报出「1611 个字节不同、最大差 255」，看似向量化严重改变了数值；但浮点数尾数最低位相差一个单位即可使某字节差达 255，该指标对 FP32 无意义。按 float32 解析后实际最大差为 $2.38e-07$ 。本表数字均以正确解析口径得出。

六个提交的比对结果（2026-09-21，全部完成）

提交	MobileNetV2 FP32	MobileNetV2 INT8	MobileNetV1 FP32	MobileNetV1 INT8	判定
#6	2.38e-07	0	1.16e-11	0	四项全过
#1	5.36e-09	0	3.58e-12	0	四项全过
#4	1.19e-07	0	3.87e-12	0	四项全过
#3	1.19e-07	3 LSB	1.96e-12	0	一项临界超出，见下
#7	9.58e-01	0	1.00e+00	0	两个 FP32 功能性失效
#2	2.38e-07	159 LSB	5.80e-12	255 LSB	两个 INT8 功能性失效

判据取官网各题详情页：FP32 相对误差 $\leq 1e-5$ ，INT8 差值 ≤ 1 LSB。表中为该提交标量档（rv64gc）与向量档（rv64gcv）对同一输入的输出张量最大差值，标量档即「同提交的参考实现」。

#7 与 #2：独立复现了已知缺陷。 本方法与模型级判据所用证据完全不同——模型级是「万张图像的 Top-1 与 x86 参考比对」，本方法是「同一提交内标量实现与向量实现对同一输入的输出张量逐元素比对」，既不依赖外部参考值，也不涉及分类正确率。两者的失败范围精确吻合：#7 坏在 FP32、两个 INT8 逐位相同；#2 坏在 INT8、两个 FP32 差值在 $1e-12$ 至 $1e-7$ 量级。这使两项结论的证据强度显著提升，且选手可在自己的环境中以同样方式独立复现。

#3：性质与上述两者截然不同，须分别对待。 其 MobileNetV2 INT8 的差异分布为——1001 个输出元素中，422 个完全相同、516 个差 1 LSB、61 个差 2 LSB、仅 2 个差 3 LSB；差异位置的标量档取值分布在 29 至 218 之间。作为对照，#2 的同一模型为 977 个元素不同、最大差 159 LSB (uint8 满量程为 255)。

上游行为比对（09-24 自查，按 P2601 I420 先例）：另以六个提交各自 fork 点的上游 LiteRT 原版为参考（标量档）与各提交向量档比对，24 组中 5 组超阈值者与上表所提提交、所涉模型、数值完全相同，19 组通过者差异均在 $1e-12$ 至 $1e-7$ 量级——S2602 无 I420 类「移植底座漂移」问题，三家有缺陷者（#7/#2/#3）问题均在向量化本身。该比对使 #7 与 #2 的证据链闭合为三条独立方法、数值互相咬合。详见《A210 卷》S2602 上游行为比对一节。

即 #3 属临界超出：绝大多数元素在 1 LSB 容差之内，仅 2 个元素超出 2 个单位，与量化路径中舍入次序的微小差异相符，不是功能性失效；其模型级精度为 6/6 全部达标，Top-1 未受影响。按官网条款「算子级输出（INT8）差值 ≤ 1 LSB」逐字执行，该项不通过。

本报告不代为判定，提请评判组裁定（新增，见第五章）：1 LSB 判据是否按字面逐元素执行。该裁定直接影响 #3 在本题的名次——其目前精度 6/6、时延 3/6，位列第二。

五、须评判组裁定的事项

共 14 项，其中 6 项直接决定提交存亡。完整数据与各口径后果见 本卷第二部分，此处仅列题目。

决定存亡的 6 项：S2605 的 +30% 门槛口径（严格口径实测无人通过，宽口径四种组合中仅一种可使 #3 通过）；S2603/S2604 的 RVV 覆盖分母（按字面执行则 S2604 上限 40%，永远达不到 70%）；S2605 #1 的 Zbc 崩溃定性；S2603 #2/#6 的 LTO 处置；P2601 #1 的并行库是否计入 130 条分母；「默认构建启用」对默认关闭、文档写明开关的提交如何认定。

规则口径与流程 7 项：S2605 #1 的 crc32c 存在 RVV 路径但在验证平台崩溃是否记为覆盖（门槛为 3/3 而该提交另有一项完全缺失，故不改变其达标与否）；AI 辅助占比 90% 的认定；5 条公开通报草稿的发送审批；S2602 算子级精度的对比基准（备忘录指向的文件无此内容，属规则缺口）；RocketMQ 的 3 轮中位数适用性与 P99 取得方式；评审人定名与 S2602 #7 所属队伍的队名补齐（同项的环境定版已于 09-20 完成，见 本卷第四部分）；S2604 #4 的说明文件口径。

已决事项 12 条，含 09-18 的「每模型取较优线程数」、09-19 的「先严后宽」、09-20 的 S2602 分母定稿为 98 项与验证平台环境定版。完整列表见 本卷第二部分 第四章，依据载于 DECISIONS_PENDING.md（内部文件，不随本次交付）。

六、结论

1. **可提交评委审核的部分：**S2604 与 S2603 两题数据完整、结论明确（S2603 #4 的性能已于 09-20 补齐），全员未达 +30% 门槛的原因有客观的热点分析支撑，且 S2603 #4 的增益分布与其向量化手段互相印证（第 2.3、2.6 节）；P2601 的正确性结论完整，含 VLEN 三档验证。两题的结论均不依赖任何待裁定事项——S2603 #2/#6 的 LTO 处置只影响这两个提交本身，不改变「本题无人达标」。

2. **尚不宜定论的部分**：S2602 的模型级数据虽已完整（六个提交全有精度与时延），但算子级精度这项明文要求**尚在执行中**（原定的上游 kernel 单测路径经查在本版本 LiteRT 上不可用，已改走同提交标量档与向量档的输出张量差分，见第 4.4 节），#2 的精度失败已于 09-21 定性为代码缺陷（以其声明的编译器大版本 gcc-15 重建后原样复现）；S2605 的 RocketMQ 压测**已完成 1 轮 60 分钟长稳测试，全部判据通过**——两消费组 Consume Diff Total 均为 0、无 OOM/Corruption/Background error、发送零失败；P99 因官方工具不输出而如实声明不可得。尚待裁定 3 轮中位数是否适用。#1 仍无正确性与性能数据。
3. **结论的可信度已作分层标注**：经隔离实验证实者（S2603 #4 的 Prefetch 归因、S2605 的 options_settable_test、P2601 #2 的段错误范围）、经多方交叉对照者（S2602 #7 的 FP32 失效）、以及单点观察未作对照者，在各详细报告中分别注明。
4. **数据层已经独立复核**：2026-09-20 以另写脚本从原始输出重算，覆盖 S2605 基线 3 项、S2605 性能 15 项、S2602 精度 30 项（原始 JSON 与汇总表零不一致）、S2602 时延 5 个提交、LX5000 场景中位数 24 项，与报告数字逐项相符。复核中发现两处文档转述错误并已更正：S2602 #3 的精度实为 6/6 而非「待补」；#2 的 EfficientNet-Lite0 FP32 时延应为 230.0 ms（四线程）而非 393.2 ms（单线程，系四线程数据到齐前的过时值）。
- 当日新产出的数据同样经独立重算：S2603 #4 的八个场景，绕开流水线汇总表、直接从 memtier 原始输出文件重新计算三轮中位数并换算对基线的差异，**八项逐一相符，零不一致**（该题增益最高的 pipeline=64 三轮原始值为 889,826.97 / 887,823.15 / 889,340.50，中位数 889,340.50，对基线 +19.1%）。
- 同日另完成三项溯源核验，均无不符：21 个上板工作树的 HEAD 与本地归档、与本报告记录三方逐一相符；本报告与 本卷第二部分 中全部 34 条永久链接（第五章附录的 22 条冻结版本链接，加正文引用的 12 条代码文件链接）经逐条请求均返回 HTTP 200，5 个仓库均为 public 可匿名访问；其中 10 条带行号的深链，其行号所指内容与文中引文逐字相符。平台环境已定版，见 本卷第四部分。
5. **局限**：
- 逐 API 的 RVV 向量化判定无法自动化，须人工核实函数体，是剩余工作的最大单项；
 - S2602 #2 在现有平台条件下无法用其声明的工具链复现，该项定性存在客观障碍；
 - 算子级精度的对比基准在组委会文件中从未定义；组委会已按「同提交标量实现」先行取数，但该口径仍须评判组确认；
 - 评审人尚未定名，代码审查的人工部分无人认领；
 - S2602 #7 所属队伍（liuyd-dev）的队名在归档中仍记为「待核实」，而该提交仍在评审流程内，结果公示与奖项发放须用到队名。
6. **建议的下一步**：将本报告与 本卷第二部分 一并提交评判组，**先行裁定 14 项口径**。其中数项将直接改变后续工作量——例如 P2601 #1 若裁定不计入分母，该提交的 130 条判定无须进行；S2603/S2604 的分母口径若确定，7 个提交的判定方式随之确定。在裁定前继续推进，存在返工风险。

七、附录：原始数据与详细报告

详细报告

关于本节所列文件：标注《…》者为本次交付的 PDF，收件人手中应有同名文件；以等宽体标注的 .md/.sh/.tsv 为组委会内部工作文件或板上原始数据，**不随本次交付**，如需查阅可另行索取。

- 《LX5000 卷》第一部分：S2603/S2604/S2605 编译与正确性
- 《LX5000 卷》第二部分：同上三题性能，含边界带复核
- 《LX5000 卷》第三部分：S2605 make check 补测与失败归因
- 《LX5000 卷》第四部分：S2603/S2604 热点函数表（§ 5.2 分母）
- 《A210 卷》第一部分：P2601 完整验证，含 VLEN 三档
- 《A210 卷》第二部分：S2602 完整验证，含六提交的精度与时延
- 《A210 卷》第三部分：A210 双板性能与环境差异，含 6 处环境修复
- 本卷第四部分：三台验证机器的环境定版快照、冻结 SHA 三方核验、影响后续排期的环境约束
- 本卷第三部分：RVV 覆盖率事实表
- 本卷第二部分：14 项待裁定事项的完整数据与口径后果
- DECISIONS_PENDING.md（内部文件，不随本次交付）：待决清单与已决事项依据
- review/discussion_drafts.md（内部文件，均未发出）：5 条公开通报草稿，均未发出
- review/formal_review.md（内部文件，不随本次交付）：形式审查结果
- 《LX5000 卷》第五部分、《A210 卷》第四部分：基线与精度参考
- 本卷第五部分：各题 Neon 算子分母表，为 RVV 覆盖率分母的唯一出处
- ACCEPTANCE_MEMO.md（内部文件，不随本次交付）：验收备忘录，本报告所引 § 4.B/ § 4.C/ § 4.D/ § 4.E、§ 5.1/ § 5.2 均出自此
- verify/build_all.sh：各提交的实录构建命令（SPECS 数组，含每条命令的文档出处标签）

板上原始数据

- LX5000：~/rvspoc-verify/（编译与正确性）、~/rvspoc-perf/（性能与热点采样）、~/rvspoc-makecheck/（make check）
- A210 b1：/work/（P2601 构建、测试、VLEN 三档、benchmark）
- A210 b2：/work/s2602/（S2602 构建、精度、时延、张量预处理块）

选手提交的冻结版本

各提交的永久链接与冻结 SHA 对照表见 本卷第二部分 第五章附录，全部 SHA 与 pr-archive/MANIFEST.tsv 逐一核验一致。

关键校验值

- 预处理张量块 blob_sha256：
f4f03ab2376a8da0e6c9bf8920afc45861ab46ba525d7330e8e150f75d4bc1d3
- 图片清单 list_sha256：c7139aafeld3eb76f5be150461dde3bf551bbbd3e127c8b03616191ddfb2c4c0
- 全部工作树的冻结 SHA 与 pr-archive/MANIFEST.tsv 一致

第二部分 待评判组裁定事项

日期：2026-09-22 编制：组委会验证侧 配套：《RVSPOC 2026 复现验证与评审阶段性报告》

适用范围：22 个冻结提交中的 21 个有效提交（S2604 #4 已于 09-01 判无效）

版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

结论先行：需评判组裁定的事项共 14 项。其中 **6 项直接决定提交是否通过**（2.1 至 2.6），其余 8 项为规则口径与流程。2.7 虽与 2.3 同涉 S2605 #1，但门槛为 3/3 而该提交另有一项完全缺失，故无论如何裁定都不改变其达标与否，已在该节注明。本文档只陈述事实与可选口径，并列出各口径分别导致的判定结果，不含组委会的倾向性意见。有两项属规则文本自身的缺口而非执行争议：S2602 算子级精度的对比基准在组委会文件中从未定义；S2602 #2 声明的编译器在验证平台不可得，致其无法按声明工具链复现。在 14 项裁定落地前，S2602、S2605 两题无法给出最终结论，P2601 与 S2603/S2604 的代码审查判定亦无法启动。

引用体例：本文所引选手代码均给出指向**冻结 SHA** 的永久链接，即评审基准的那一版，不随分支后续变动。规则原文逐字引用，出处标注到节号或行号。

一、事项总表

序	事项	影响范围	性质
2.1	S2605 的 +30% 性能门槛口径	S2605 全题 4 个提交，其中 3 个有性能数据（#1 因崩溃无法测）	决定存亡
2.2	S2603/S2604 的 RVV 覆盖率分母	7 个提交	决定存亡
2.3	S2605 #1 的 Zbc 崩溃定性	1 个提交	决定存亡
2.4	S2603 #2/#6 的 LTO 构建失败处置	2 个提交	决定存亡
2.5	P2601 #1 的并行库路线如何计分	1 个提交	决定存亡
2.6	「默认构建启用」对默认关闭者的认定	2 个提交	决定存亡
2.7	S2605 #1 的 crc32c 是否记为覆盖	1 个模块	规则口径（不决定存亡，见该节）
3.1	AI 辅助占比 90% 的认定	3 个提交，口径影响 5 题	规则口径
3.2	5 条公开通报草稿的发送审批	6 个提交	流程，时效最强
3.3	S2602 算子级精度的对比基准	S2602 全题	规则缺口
3.4	RocketMQ 压测的两项执行口径	S2605 全题	规则口径
3.5	评审人定名，及 S2602 #7 所属队伍的队名补齐（环境定版已于 09-20 完成）	全局	流程
3.6	S2604 #4 的说明文件口径	1 个提交，已判无效	规则口径，仅留档
3.7	S2602 算子级 1 LSB 判据是否按字面执行	1 个提交的名次	规则口径

二、S2605 与 S2603 的六项存亡裁定，及一项相关口径（2.1-2.7）

2.1 S2605 的 +30% 性能门槛口径

须裁定两点：必测三项的组合方式；边界值的取数方式。

实测数据（对组委会标量基线，《LX5000 卷》第二部分）：

提交	fillrandom	readrandom	seekrandom
----	------------	------------	------------

#3	+4.6%	+29.9%（六轮合并）／+32.6%（复核轮）	+31.7%
#2	+7.2%	-1.7%	-4.7%
#4	-6.7%	-14.0%	-18.2%

严格口径（必测三项全部 $\geq +30\%$ ）实测结果：三个提交均不通过。

转宽口径后四种组合的后果：

组合	#3 的结果	#2 与 #4	本题结果
平均 $\geq 30\%$ ，取六轮合并	+22.1%，不通过	不通过	无人通过
平均 $\geq 30\%$ ，取复核轮	+23.0%，不通过	不通过	无人通过
多数达标（2/3），取六轮合并	1/3，不通过	不通过	无人通过
多数达标（2/3），取复核轮	2/3，通过	不通过	#3 通过

须知的事实：#3 的 readrandom 三次测量分别为首轮 +29.2%、复核轮 +32.6%、六轮合并 +29.9%，其轮间波动幅度大于该值与 30% 门槛的距离。采多数口径时，判定结果取决于对该值的取数选择；采平均口径时两种取数结论一致（+22.1% 与 +23.0%，均低于 30%）。

上述数字已于 09-20 以独立脚本从 db_bench 原始输出重算，与报告逐项相符；所用基线（fillrandom 244,774、readrandom 292,356、seekrandom 141,371 ops/sec）亦经同法复核一致。

2.2 S2603/S2604 的 RVV 覆盖率分母

须裁定：§ 5.2 的「取 Top 20 热点函数为分母，向量化比例 $\geq 70\%$ 为过」如何执行。

实测数据（《LX5000 卷》第四部分，标量版在验证平台的 perf 采样）：

场景	自有代码	不可向量化项构成
S2603 base	16/20	系统调用 3 项、libc 字符串 1 项
S2603 pipeline=16	14/20	系统调用 3 项、libc 字符串与内存 3 项
S2604 base	8/20	锁原语 5 项、系统调用 3 项、libevent 2 项、libc 2 项

S2604 的首位热点为 pthread_mutex_lock，占 14.26%；锁原语合计 25.4 个百分点，系统调用合计 19.7 个百分点，自有代码合计仅 17.8 个百分点。

关键事实：S2604 的 Top 20 中自有代码仅 8 项，占 40%。按字面执行分母，即便选手将全部自有代码向量化，覆盖率上限亦为 40%，永远无法达到 70%。

口径	后果
字面执行，Top 20 原样为分母	S2604 三个提交必然全部不通过；S2603 上限约 70–80%
分母只取自自有代码	S2603 按 16 项（base）或 14 项（pipeline）计，S2604 按 8 项计
按自有代码热点的时间占比加权	须另行定义加权规则

补充事实：S2603 #2 与 #3 自行替换了 memchr、memcpy 等 libc 函数，而这些函数本身位于 Top 20 分母内。采「分母只取自有代码」口径时，该部分工作将被排除在分母之外，亦即不计入其覆盖率。

2.3 S2605 #1 的 Zbc 崩溃定性

事实：该提交使用 Zbc 标量 clmul 指令，而 LX5000 的 ISA 不含 zbc，DB::Open 即收到 signal 4 崩溃，db_test 无法运行。崩溃点经 gdb 定位至 util/crc32c_riscv.cc:66 的 CrcClmulLong，出错指令为 clmul a5,a5,a4，调用链为 DB::Open → DBImpl::Recover → log::Writer → crc32c_riscv。崩溃函数原文 (util/crc32c_riscv.cc#L65-L68)：

```
inline uint32_t CrcClmulLong(uint64_t msgpoly) {
    uint64_t tmp = Clmul(msgpoly, kBarrett1);
    return static_cast<uint32_t>(Clmulr(tmp, kBarrett2));
}
```

其代码注释明示路径选择为编译期决定、无运行时检测 (util/crc32c.cc#L1119-L1122)：

```
#elif defined(HAVE_RISCV_CRC32C)
    // RISC-V with the Zbc carryless-multiply extension: always available at
    // runtime because selection is compile-time (-march=...zbc...).
    return ExtendRISCVImpl;
```

赛题规则原文 (ACCEPTANCE_MEMO.md (内部文件，不随本次交付) § 4.C 转录)：「验证平台：蓝芯 LX5000 开发板」；产出要求含「能在验证平台正确运行 RocksDB 核心测试集 (db_test)」。

选手说明文件称在「挑战指定的 Spacemit K3」开发板构建与验证。K3 支持 Zbc，故其本地验证不会暴露此问题。

口径	后果
判正确性不达标	该提交本题出局
以其标量回退构建重测继续评审	组委会已验证以其文档中的 PORTABLE=rv64gc 重编后 MockEnvTest 通过；但其余提交均在 LX5000 原生验证，须考虑口径公平性

2.4 S2603 #2/#6 的 LTO 构建失败处置

事实：二者在默认构建 (Redis 的 -flto=auto) 下于 LX5000 编译失败，报 lto1: fatal error: target specific builtin not available。仅追加 -fno-lto 其余不变，两者均编译通过，代码本身无误。机制为 LTO 在链接期以基线 -march 重新生成代码，向量扩展的 builtin 随之失效。选手文档未写明 LTO 处理方式，亦未写明最低编译器版本。

对照：S2603 #3 的说明文件预见了此问题，为 RVV 对象单独加 -fno-lto 隔离，构建正常。

性能条件组数据 (加 -fno-lto 后测得，仅备用，采信与否取决于本项裁定)：

提交	八个场景的表现
#2	全部不高于基线，区间 -7.9% 至 +0.1%
#6	无持久化的七个场景最好为 pipeline=64 的 +16.5%；开启 AOF 后服务端断言崩溃

#6 的崩溃信息为 Guru Meditation: You tried to retain an object allocated in the stack (object.c:637)，调用链为 readQueryFromClient → processCommand → call → alsoPropagate。

机制已于 09-20 逐行核实，因果链完整：该提交为 GET/SET 的键做了零拷贝优化——在查询缓冲区内就地伪造一个 SDS 头，再把缓冲区中的字节直接借用为字符串对象（src/networking.c#L3509-L3518）：

```
int borrow_key =
    (pcmd->argv[0] == shared.get && c->multibulklen == 1) || ...;
if (borrow_key && c->bulklen <= 31) {
    arg_data[-1] = SDS_TYPE_5 | (c->bulklen << SDS_TYPE_BITS);
    arg_data[c->bulklen] = '\0';
    initStaticStringObject(pcmd->borrowed_key, arg_data);
    arg = &pcmd->borrowed_key;
}
```

其中 arg_data 即 c->querybuf + c->qb_pos，而 initStaticStringObject 将该对象的引用计数置为 OBJ_STATIC_REFCOUNT（src/server.h:1111）。该对象随后进入 pcmd->argv[]。开启 AOF 后命令须向 AOF 传播，alsoPropagate 对参数对象调用 incrRefCount，而 incrRefCount 在遇到 OBJ_STATIC_REFCOUNT 时即抛出上述 panic（src/object.c:632-640）：

```
} else if (o->refcount == OBJ_STATIC_REFCOUNT) {
    serverPanic("You tried to retain an object allocated in the stack");
}
```

此缺陷仅在 AOF 开启时暴露，正是其因 LTO 编译失败而未能运行回归测试所漏掉的问题。

口径	后果
组委会加 -fno-lto 继续评审	与 #3 选手自行处理的方式等效；但 #6 仍须先过正确性关，预期 runtest 的 AOF 相关用例会命中上述崩溃
判可复现性不达标	二者不进入后续环节

2.5 P2601 #1 的并行库路线如何计分

事实：该提交不修改上游 Arm 源码，RVV 实现独立置于 riscv/library/，自带 CMakeLists、toolchain 与 smoke 测试，其 riscv/README.md#L12 原文为：

Upstream Arm sources are not modified; everything new is added here so the upstream tree stays diff-friendly.

由此产生三个连带事实：

事项	事实
分母计入	分母为 KleidiCV kleidicv.h 暴露的 130 个 C API 入口。争点在于本提交的算子是否计入该分母——其实现不在上游树内，而在自建的 riscv/library/，若不计入，则本提交的 130 条判定均无须进行
证据强度	§ 4.C 判据为「KleidiCV 自带测试套件（gtest）通过」，但该套件在技术上无法施加于本提交——其代码接不入上游树。组委会只能运行其自写的 8 项 smoke 测试（每项输出两行），而 #3、#4 各运行 4496 个用例
数据完整性	§ 4.D 考察 benchmark 数据完整性。本提交产出 9 条测项；同题另三家为 #2 的 381 条、#3 与 #4 各 372 条，均在同一量级

该提交的 RVV 代码本身在 VLEN 128、256、512 三档下全部通过，无质量问题。争点在于该路线是否符合「移植 KleidiCV」的题意，以及由此导致的横向可比性缺失。

2026-09-23 补充：本题另有一项新增证据可供排名参考——I420/YV12 布局专项（见本卷第一部分 2.1 节）显示 #4 的解码与原版 26.03 布局完全一致，而 #3 在两组解码中分别失败 96 例与 80 例；编码布局一项两家同样失败。该证据由评委独立提供、组委会已用自有归档源码复现且计数吻合。**评委已于 09-22 给出倾向 #4 的意见**（理由：I420 解码与原版一致、代码组织更清晰、#3 的优势不足以抵消原版兼容性问题），详见《A210 卷》。该意见尚待评判组合议确认；**若据此启用《验收备忘录》§ 5.3 的综合比评（而非时间优先），须写明启用理由。**

2.6 「默认构建启用」对默认关闭者的认定

事实：§ 5.1 的判据为「某算子存在 `__riscv_vector` 路径且在默认构建中实际启用」。实测两个提交属「文档写明开关、但默认关闭」：

提交	机制	默认状态
S2603 #6	<code>ifeq (\$(BUILD_RVV),yes)</code> ；不加该开关则走 <code>rvv_optim.h</code> 的标量别名	关闭
S2604 #2	<code>--enable-riscv-rvv configure</code> 开关	关闭

S2603 #6 的门控原文（[src/Makefile#L388-L392](#)）：

```
ifeq ($(BUILD_RVV),yes)
REDIS_SERVER_OBJ+=rvv_optim.o
FINAL_CFLAGS+=-DREDIS_RVV
RVV_CFLAGS?=-march=rv64gcv -mabi=lp64d
```

对照：S2603 #2 为 `RVV?=yes`，默认开启，且运行时另有 `getauxval(AT_HWCAP)` 分派；P2601 #2、#3、#4 在 `riscv64` 上默认开启。

组委会构建时按选手文档带了相应开关，实录载于 `verify/build_all.sh` 的 `SPECS` 数组。（须说明：初期曾有四例未严格照文档构建，均已更正，见阶段性报告第三章；此处所述为更正后的状态。）按字面执行该判据，上述两个提交的 `RVV` 实现将被扣除。

2.7 S2605 #1 的 `crc32c` 是否记为覆盖

事实：S2605 的 Neon 分母仅 3 个模块（本卷第五部分 § 3），90% 门槛实际等于 3/3 全覆盖。各提交的覆盖情况：

提交	<code>crc32c</code>	XXPH3	XXH32/64	覆盖数
#3	向量 <code>vcmlul/vcmlulh</code> ，平台支持	有，默认启用	有	3/3
#4	向量 <code>vadd/vand</code>	有，默认启用	有	3/3
#1	有，但使用 <code>Zbc</code> 标量 <code>clmul</code> ，平台不支持、运行即崩溃	有，默认启用	无	2/3 或 1/3
#2	有	无	无	1/3

须裁定：#1 的 `crc32c` 模块「存在 `RVV` 路径」但「在验证平台不可运行」，是否记为覆盖。本项与 2.3 相关但可独立裁定。

本项不决定 #1 的存亡：门槛为 3/3，而 #1 的 `XXH32/64` 一项完全无 `RVV` 实现，故无论 `crc32c` 记或不记，#1 在本门槛项上均不达标（2/3 或 1/3）。裁定影响的是记录口径与本题的横向比较尺度，以及同一判据将来施于其他题目时的一致性。决定 #1 存亡的是 2.3（`Zbc` 崩溃如何定性）。

三、规则口径与流程（3.1-3.6）

3.1 AI 辅助占比 90% 的认定

事实：velonica0（GitHub 账号，对应队名「RVV 智能」，本文档其余各处均以队名出现）在三个提交（P2601 #2、S2604 #1、S2605 #1）中声明 AI 辅助占比约 90%，并说明「优化方案决策、关键代码实现、编译、真机运行、性能结果核对以及最终提交均由作者完成并复核」。规则要求「说明使用方式及占比」，未设占比上限；赛题注意事项另有「须自主实现」「禁止直接搬运」条款。

影响面：该队五个赛题各投一个提交，是投递面最广的两队之一（另一队为 xiazhuzhao／260818，同为 5 个提交）。两队均未在同一赛题内多投，故不触发 09-01「同队多 PR」条款。该队的五个提交为 P2601 #2、S2602 #2、S2603 #2、S2604 #1、S2605 #1，其中三个已声明 90% AI 辅助，故本项口径一经确定，影响面覆盖五个赛题。

三条 AI 说明询问已于 09-04 发出并于 09-07 闭环，答复内容载于 review/formal_review.md（内部文件，不随本次交付）。

3.2 5 条公开通报草稿的发送审批

事实：草稿已备于 review/discussion_drafts.md（内部文件，均未发出），全部未发出。选手需要回复时间，发出越晚，压缩申诉窗口前的沟通时间越多。

序	对象	主题
1	S2603 #2、#6	LTO 构建失败
2	S2603 #4	15 项测试失败（stderr 触发 Tcl exec 判错；Zicbop 编译期宏误禁 Prefetch）
3	S2605 #1	Zbc 崩溃，含指令级定位
4	P2601 #2	Remap FP32 段错误，及标量档链接失败
5	S2602 #1	EfficientNet-Lite0 INT8 时延 3240 ms

未起草的一项：S2602 #2 的 MobileNet INT8 精度归零。原因见 3.3 下一条与本文 2 段说明——其声明的编译器在验证平台不可得，证据未闭合，不宜据此通知选手。

3.3 S2602 算子级精度的对比基准（规则缺口）

事实：ACCEPTANCE_MEMO.md（内部文件，不随本次交付）§ 4.C 要求 S2602 做「模型级加算子级精度」，算子级标准为「FP32 相对误差 $\leq 1e-5$ 、INT8 ≤ 1 LSB」，并称「参考值、判定区间、评测口径见《A210 卷》第四部分」。该文件通篇仅含模型级内容，无任何算子级定义。

此外，x86 参考仅有 6 个模型级 Top-1 数字，无逐算子张量转储，技术上无法作为算子级基准。

可行的唯一选项是以「同提交的标量参考实现」为基准（LiteRT 的 reference_ops，即 -march=rv64gc 构建），与 P2601「对照 KleidiCV 参考实现」的口径对称。

组委会已调研出执行路径：启用上游的 TFLITE_KERNEL_TEST=ON，其 137 个 kernel 单测本身即「优化实现对 reference_ops」的逐元素比对，且选手不可篡改阈值；配合 QEMU 在 VLEN 128、256、512 三档运行。口径一经确定即可执行，约 2 人日加 1-2 个板夜。原定路径经查在本版本 LiteRT 上不可用，已改走替代路径；本项仍待口径裁定：

- b2 原未装 qemu-riscv64，已补装并实测 VLEN 128/256/512 三档均可用；
- 上游 TFLITE_KERNEL_TEST=ON 在本版本 LiteRT 中本身编不过，组委会定位出五处上游自身的缺陷。前四处已处置，第五处无法绕过（详见阶段性报告第 4.4 节）。前三处为仓库布局漂移，已

以符号链接补位（**不编辑任何文件**）： LiteRT 独立仓库根下无 TF 单体仓库的 compiler/ 目录； kernels/CMakeLists.txt 引用的 tools/versioning/op_version.cc 在本版本已移至 converter/ tools/versioning/； XLA 的 xla/tsl/platform/default/logging.cc 已上移一级。第四处为条件编译缺失： kernels/CMakeLists.txt 无条件把 tools/delegates/ xnnpack_delegate_provider.cc 列入测试框架源，而该文件全文无任何守卫宏，在 TFLITE_ENABLE_XNNPACK=OFF 时仍调用 XNNPACK API 而编译失败，故 kernel 测试在本版本必须开启 XNNPACK 方可构建。六个提交对这四处均无改动（xnnpack_delegate_provider.cc 的 md5 六者完全相同），故该缺陷与选手无关，补位方式亦将对六者一视同仁。开启 XNNPACK 不影响被测对象—— kernel 单测直接调用算子实现，不经由 delegate 分发。

- 补位后 configure 通过，169 个 kernel 测试目标成功注册；但构建时命中第五处缺陷—— 测试框架的 quantization_utils.cc 传递依赖 TensorFlow core 的 protobuf 生成头（types.pb.h），而 CMake 构建路径从不运行 protoc 生成它们（那是 Bazel 的职责）。补齐它等于在 CMake 路径上重造 TensorFlow 的一部分构建系统，代价与方法学风险均不相称，故终止该路径。

改用的路径：同一提交分别以 -march=rv64gc（标量）与 -march=rv64gcv（向量）构建，对同一输入 dump 输出张量，逐字节比对；标量档即「同提交的参考实现」。该口径取自选手 #4 自带的 scripts/compare-model-differential.py，覆盖 VLEN 128/256/512 三档，是唯一把 § 4.C 的 1e-5 与 1 LSB 写进断言的工具。采用选手自己的方法学，可最大限度减少口径争议。

故本项裁定所决定的是「该差分比对是否足以满足 § 4.C 的算子级精度要求」。组委会已先行开跑取数（截至 09-21 #6 的双档在构建中），裁定后再定性。

另一项相关的客观障碍：S2602 #2 在 PR 描述中声明编译器为 GCC 15.2.0，而其代码使用 __RISCV_VXRM_RDN（GCC 14 才引入的定点舍入模式常量），故 gcc-13 构建失败。用法原文（tflite/kernels/internal/optimized/rvv_quantization_utils.h#L37）：

```
return __riscv_vnclip_wx_i32m1(product, 0, __RISCV_VXRM_RDN, vl);
```

GCC 15 在验证平台（Debian 13 trixie riscv64）的软件源中不可得。唯一可用于构建该提交的编译器为 gcc-14。以全新构建树重做验证（每项 500 张，同一 harness、同一批张量、同一模型文件）：#2 的库在 MobileNetV2 INT8 为 0.0%、MobileNetV1 INT8 为 1.4%、EfficientNet-Lite0 INT8 为 73.6%；作为对照，#6 的库在同批图像上分别为 68.4% 与 69.8%。万张全量测试中该提交的三个 FP32 模型与 x86 参考完全一致。

该失败在全新构建树下稳定复现，并非组委会的构建产物问题，且范围精确限于 MobileNet 系的 INT8 路径。但由于其声明的 GCC 15.2.0 不可得、gcc-13 又无法构建，不能排除系 gcc-14 与 gcc-15 的代码生成差异所致。

3.4 RocketMQ 压测的两项执行口径
须裁定两点：

序	问题	事实
1	§ 4.D「每项 3 轮取中位数」是否适用于 60 分钟稳定性测试	字面执行为 3 小时乘 3 个提交，合计 9 小时纯压测
2	RocketMQ 的 P99 如何取得	官方 benchmark 工具不输出 P99。S2605 #3 已在其 docs/COMPLIANCE.md 公开承认该限制，#2 的数据表中亦仅有 Average 与 Max RT。须自写外部采样器（约 0.5 人日），或降级为「如实声明不可得」

赛题原文 (rvspoc/blog/2026-06-17-01.md 第 21 行):「本赛题要求参赛队伍将 RocksDB v11.1.1 完整移植至 RISC-V 64 平台……且能成功以 JNI 动态库的方式部署到 RocketMQ 5.5.0 运行环境。」

补充事实: 受测范围为 #2、#3、#4 三个提交, #1 因 DB::open 崩溃而排除。三者的 JNI 集成方式无法完全统一: #2 与 #3 的仓库中缺 RemoveConsumeQueueCompactionFilter, 而 RocketMQ 5.5.0 硬依赖该类, 故二者须退回其文档自述的集成方式 (#2 注入 .so 至 fat JAR, #3 整包替换 bin-release), #4 则可走从源码构建的完整路径。该差异本身即「JNI 集成完整度」的评审素材, 须记入评审记录。

3.5 两处定名空缺与 A210 环境定版

其一, 评审人未定名。备忘录 § 7 的 A、B、C、D 四位评审人仍为代称, 实名未定, 代码审查的人工部分无人认领。备忘录 § 7 另规定「复核独立核对关键数据 (至少抽验性能中位数与正确性结论)」, 该角色至今空缺。

其二, 一支选手队伍的队名至今空缺。pr-archive/MANIFEST.tsv 中 liuyd-dev 的队名栏记为「待核实」, 其余八支队伍均有队名。该账号有两个提交: S2604 #4 (已于 09-01 判无效) 与 **S2602 #7**——后者为有效提交, 且是该题时延最快者 (六模型全部达标、平均 55.8 ms), 虽因三个 FP32 输出为噪声而精度仅 3/6, 仍处在评审流程之内。**结果公示与奖项发放须用到队名, 此项须在出具最终结论前补齐。**

环境定版快照已于 09-20 补做, 覆盖 LX5000 与 A210 双板, 见本卷第四部分。本项须裁定的只剩评审人定名一事。

定版同时查明两板并非同一镜像: b1 有效主频约 2.29 GHz, b2 锁定 1.896 GHz, 性能差 12% 至 30%, 故严禁跨板比对; OpenSSL 版本不同 (3.5.6 对 3.5.1); b1 的 /home 为独立的 26 MB 分区; ISA 通告亦不同 (b1 未通告 bitmanip), 但硬件一致, 系设备树差异。组委会已修复 6 处开箱即用问题, 详见《A210 卷》第三部分 第六节。

定版另暴露三项影响后续排期的约束, **其中两项已解除:** b2 的 qemu-riscv64 已于 09-20 补装并实测 VLEN 128/256/512 三档可用; b2 原缺的裸 gcc/g++ 已于 09-21 补装。**仍存在的一项:** b2 工作区余量偏紧 (09-20 定版时 8.7 GiB), 与「不删构建树」的操作纪律存在冲突, 须在全量取数前扩容或清理。b1 未装 gcc-13 一项暂不影响, 因 P2601 四个提交均以 gcc-14 构建。详见本卷第四部分 第三章。

3.6 S2604 #4 的说明文件口径

说明写在 PR 描述 (8,769 字节, 章节齐全) 而非仓库文件。规则原文为「须同时附带一份说明文件」。该提交已因创建时间晚于冻结点判无效, 本项仅作口径留档, 不影响成绩。

3.7 S2602 算子级 1 LSB 判据是否按字面逐元素执行

须裁定: 官网 S2602 详情页载明「算子级输出 (INT8) 差值 ≤ 1 LSB」。该判据是否按字面逐元素执行, 即只要有任一元素差值超过 1 LSB 即判不通过。

事实: 算子级差分比对中, S2602 #3 的 MobileNetV2 INT8 出现最大 3 LSB 的差异。其分布为——1001 个输出元素中, 422 个完全相同、516 个差 1 LSB、61 个差 2 LSB、**仅 2 个差 3 LSB**。该提交的模型级精度为 6/6 全部达标, Top-1 未受影响。

对照: 同一判据下, #2 的同一模型为 977 个元素不同、最大差 159 LSB (uint8 满量程 255), 另一模型最大差 255 LSB; #6、#1、#4 三家的四项全部为 0 或达标。即 #3 属临界超出, 与 #2 的功能性失效性质不同。

口径	后果
按字面逐元素执行	#3 本项不通过；#6、#1、#4 通过；#7、#2 不通过
设定容忍比例（如允许少量元素超出）	须另行定义比例与上限，且该口径赛前未公开
以模型级 Top-1 达标为准、算子级仅作参考	#3 通过；但与官网明列的算子级判据不符

影响：该裁定直接影响 #3 在本题的名次——其精度 6/6、时延 3/6，目前位列第二（第一为 #6）。不影响 #6 的领先地位。

须注意的口径公平性：三种口径中，后两种均属赛前未公开的新增解释。若采用，须对六个提交一体适用，并在评审记录中写明理由。

四、已决事项

以下 12 项已有结论，列此以免重复讨论。完整依据载于 DECISIONS_PENDING.md（内部文件，不随本次交付）。

日期	事项	结论
09-01	冻结后 PR 判定	以 PR 创建时间为准；S2602 #7 有效，S2604 #4 无效
09-01	同队多 PR	按队伍认定；核实后确认 9 队无一队多 PR，条款无适用对象
09-01	P2601 排名	按最后 push 时间先后；均通过再综合比评
09-02	性能测试轮数	统一 3 轮取中位数，基线与选手一致
09-04	S2602 测试集	ImageNetV2 matched-frequency 10,000 张
09-07	对选手沟通	所有对外沟通须经审批，一次批准不外推
09-14	预编译二进制	22 个提交全部提交完整源码，§ 6 补交核验无适用对象
09-17	S2602 计时板	使用 b2，其频率可读可锁，可审计性最好
09-18	S2602 时延线程数	每模型取较优线程数
09-19	S2605 与 S2602 门槛路径	先严后宽：先按严格口径判定，若无提交通过再转整体口径
09-20	S2602 RVV 覆盖分母	定稿为 98 项，门槛 89/98；剔除 legacy_optimized_ops.h 的 9 项，已核实该文件在源码树中无任何引用
09-20	验证平台环境定版	三台机器的系统、内核、工具链与 ISA 通告已快照定版（本卷第四部分）；冻结 SHA 三方核验与永久链接可达性核验均无不符。本项对应 3.5 的后半，3.5 余下的评审人定名仍未决

五、附录：数据出处

本文所引数据均可回溯至下列文件：

- 《LX5000 卷》第二部分：2.1 的 S2605 性能数据、2.4 的条件组数据
- 《LX5000 卷》第四部分：2.2 的热点分母与构成
- 《LX5000 卷》第一部分：2.3 的 Zbc 崩溃定位、2.4 的 LTO 归因实验
- 《A210 卷》第一部分：2.5 的三项连带事实
- 本卷第三部分：2.6 的默认构建状态、2.7 的 S2605 覆盖表
- 《A210 卷》第二部分：3.3 的编译器与精度数据

- 本卷第五部分：2.7 与 09-20 分母定稿的依据
- review/formal_review.md（内部文件，不随本次交付）：3.1 的 AI 说明询问与答复
- review/discussion_drafts.md（内部文件，均未发出）：3.2 的 5 条草稿全文
- 《A210 卷》第三部分：3.5 的双板差异与环境修复
- DECISIONS_PENDING.md（内部文件，不随本次交付）：第四章已决事项的完整依据

选手提交的冻结版本（永久链接，指向评审基准的那一版）：

赛题	提交与冻结 SHA
P2601	#1 dcc5fd69 #2 9bb0aaaf #3 289ac3fb #4 c59a7eb3
S2602	#1 faf2a6dd #2 7bfae1af #3 da727af4 #4 28bd4f18 #6 dc1cede4 #7 36226ec3
S2603	#2 5a29525a #3 621fc1bd #4 82647349 #6 6dbbc85e
S2604	#1 e52d97f5 #2 7ee8609d #3 883e8764 #4 c1e9ad10 （已判无效）
S2605	#1 7847ab1e #2 1e390720 #3 fa3fd926 #4 9ec8dc8e

全部 SHA 与 pr-archive/MANIFEST.tsv 逐一核验一致，本地归档位于 pr-archive/。

板上原始数据：LX5000 的 ~/rvspoc-verify/、~/rvspoc-perf/、~/rvspoc-makecheck/； A210 b1 的 /work/； A210 b2 的 /work/s2602/。

2.1 所引的 S2605 性能与基线数字已于 09-20 以独立脚本从 db_bench 原始输出重算，逐项相符。

第三部分 RVV 覆盖率事实表

版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

生成日期：2026-09-22 对应验收备忘录 § 4.E 第 1 项、§ 5.1（90% 类）、§ 5.2（70% 类）

本文件只记录客观事实，不下判定。五个口径阻塞未决（见文末第五章），故「判定」栏一律留空，待评判组裁定后填入。

判定依据（§ 5.1 原文）：「某算子存在 `__riscv_vector` 路径且在默认构建中实际启用，即记为覆盖 1 项」——两个条件缺一不可，故本表对每个模块分列两栏。

一、S2605 RocksDB（分母 3 个模块，90% 门槛 = 3/3 全覆盖）

分母出处：本卷第五部分 § 3。RocksDB 原生 Neon 实现极少，故分母仅 3 项。

PR	队伍	① crc32c_arm64	② XXPH3 (util/xxph3.h)	③ XXH32/64 (util/xxhash.h)	覆盖数	判定
#1	RVV 智能	√ <code>crc32c_riscv.cc/h</code> （见下注）	√ 19 个 <code>__riscv_v*</code> <code>intrinsic</code> ；自动选择块含 <code>#elif defined(__riscv) && defined(__riscv_vector)</code> → 默认启用	✗ 无任何 <code>__riscv</code> /RVV 代码	2/3	待裁定

#2	zlwindy	√ crc32c_riscv.cc/h (clmul + 向量)	✗ 0 处	✗ 0 处	1/3	待裁定
#3	SNL	√ crc32c_riscv64.cc/h (vclmul/vclmulh/vget)	√ 23 个 intrinsic; 自动选择块含 riscv 分支 (附加! defined(ROCKSDB_DISABLE_RVV_XXHASH) 开关) → 默认启用	√ 34 处	3/3	待裁定
#4	260818	√ util/crc32c.cc 内 RVV 段 (vadd/vand, #ifdef __riscv_vector)	√ 23 个 intrinsic; #if defined(__riscv_vector) 参与自动选择 → 默认启用	√ 80 处	3/3	待裁定

事实注记

1. #1 的 crc32c 虽存在 RVV 路径，但在验证平台不可用：其实现使用 Zbc 标量 clmul，而 LX5000 的 ISA 不含 zbc，DB::Open 即 SIGILL 崩溃（《LX5000 卷》第一部分 § 3.3 已 gdb 定位至 util/crc32c_riscv.cc:66）。「存在 RVV 路径」与「在验证平台可运行」是两回事，该模块是否记为覆盖，需评判组一并考虑。
2. 指令族差异：#1 仅用标量 clmul (Zbc)；#2 用 clmul + 向量；#3 用向量 vclmul/vclmulh (Zvbc)；#4 用通用向量指令 (vadd/vand)。#3、#4 走的是向量路径，LX5000 支持 zvbc，不受 #1 的问题影响。
3. 三家的 XXPH3 均在 -march=rv64gcv 下自动启用，无需额外开关，满足「默认构建实际启用」。
4. 备忘录 § 5.1 特别提示：S2605 的赛题必做项（布隆过滤器位图查找、SST 序列化、CRC32C）与本 Neon 分母表是两套要求——bloom 与序列化在上游本就无 Neon 实现，不在分母内，但赛题要求向量化。评审记录须分开陈述，不可混为一谈。

实测旁证：#2、#3、#4 的 util/bloom_impl.h 均含 __riscv_vector 代码（#1 无），属赛题必做项的达成情况，不计入本表的 3 项分母。

二、P2601 KleidiCV（分母 130 个 C API 入口，90% 门槛 = 117/130）

⚠ 逐 API 覆盖率无法可靠自动判定——本节只给可核验的事实

组委会尝试了两种自动化判据，结果相差悬殊，说明该方法本身不可靠，不予采用：

判据	#2	#3	#4
宽松（API 名出现在任一 *_rvv.cpp 中）	53.1%	65.4%	62.3%
严格（实现文件须含真实 __riscv_v* 调用，按文件名映射）	37.7%	14.6%	21.5%

两者皆不可信，原因有三：

1. 三家的代码组织粒度完全不同，文件名→API 的映射是有损的：#2 为「每算子一个小文件」（约 50 个）、#3 为「每类别一个大文件」（9 个，但其 transform 类合计 565 处 intrinsic 中，transform_rvv.cpp 单文件即占 557 处）、#4 居中（18 个）。文件数之间不可比。

2. 「落在 rvv 命名空间」≠「真的向量化」——已实证：#4 的 `kleidicv/src/analysis/canny_rvv.cpp` 共 198 行、确在 namespace `kleidicv::rvv` 内，但 0 个 `__riscv_v*` 调用（纯标量实现），且被 `#if KLEIDICV_EXPERIMENTAL_FEATURE_CANNY` 守卫、该选项默认 OFF。
3. API 测试全过不等于全部向量化——#3 与 #4 的 `kleidicv-api-test` 各 4496 用例零失败，但二者含真实 intrinsic 的文件分别只有 9 个与 18 个。解释是：多数 API 落在 rvv 命名空间内的标量实现上，测试的是正确性而非向量化，标量实现当然也能通过。

结论：逐 API 的向量化判定必须人工逐条核实函数体，这与计划中标注的「最大人工项」一致。

可核验的客观事实：按算子类别的 RVV intrinsic 分布

算子类别	#2	#3	#4
analysis	34	74	96
arithmetics	68	79	77
conversions	370	343	162
filters	198	239	88
morphology	—	31	12
resize	11	135	33
transform	12	565	198
共享头 (include/kleidicv/rvv.h)	129	—	—
intrinsic 总数	822	1466	666
覆盖类别数 (共 7 类)	6	7	7

- #2 未触及 morphology 类（另两家均有），且其 transform、resize 两类的 intrinsic 数量（12、11）显著低于同侪，与其 `kleidicv-api-test` 在 `RemapF32`（transform 类）段错误崩溃、以及标量档缺 `min_max_loc`/`count_nonzeros` 符号的现象方向一致。
- #3 的 intrinsic 总量最高（1466），且 7 类全覆盖。
- #4 采命名空间宏替换（`config.h.in:68` 将 baseline 命名空间在 riscv64 上定义为 `rvv`），故其 `rvv` 命名空间内既有真向量化实现、也有标量实现（`canny` 即一例），人工核实时须逐文件区分。

#1 未纳入

采「不改上游、另起并行库 `riscv/library/`」路线，其算子是否计入本 130 条分母属规则解释，未裁定前不启动（见《A210 卷》第一部分 § 3.1 与文末第五章第 2 项）。

三、S2602 LiteRT（分母已定稿：98 项，门槛 89/98）

3.1 「约 105」的模糊来自计数口径错误

原表按函数名列举，但跨文件同名函数会被误合并——`Run`、`AveragePool`、`MaxPool`、`MulElementwise`、`RoundToNearest`、`Tanh`、`Logistic`、`LstmCell` 等 12 个名字在多个头文件中各有独立实现。按函数名去重得 92，低估了真实实现数。正确计数单位是（文件，函数）对。

口径	分母	90% 门槛
（文件，函数）对，含 <code>legacy_optimized_ops.h</code>	107	97/107

(文件,函数) 对, 不含 legacy ← 建议采用	98	89/98
按函数名去重 (错误口径, 会合并不同实现)	92	—

3.2 legacy_optimized_ops.h 的 9 项应予剔除——依据是事实而非主张

选手 #7 在其 tools/rvv_mock/audit_rvv_coverage.py:18 声明该文件为「dead legacy path, not reachable from the current interpreter」。组委会已核实该陈述属实：

```
grep -rl "legacy_optimized_ops.h" tflite/ --include=*.cc --include=*.h
→ 除其自身外，无任何文件引用
```

§ 5.1 的判据要求「在默认构建中实际启用」。一个无人 include 的头文件永不参与构建，其中的 Neon 实现既不会被执行、也不合理要求选手移植。故剔除，分母定为 98。

3.3 ruy 的 6 个 ARM 内核单列

按 ACCEPTANCE_MEMO.md:35，ruy 作为外部依赖单列为独立评分项，不并入 98 项计百分比。

须澄清的文档矛盾：该条在备忘录 § 2 写成已定事项，但 REMAINING_WORK.md:73 写「口径待确认」，而其所引的待决条目正文并未提及 ruy。列为第五章第 5 项。

3.4 逐提交判定

未启动 —— 分母已定稿（98 项），588 条判定待口径裁定后进行。鉴于 P2601 的经验（自动化判据在两种严格度下结果相差 3 倍，不可靠），本题同样以「按模块的 intrinsic 分布 + 人工抽查」为主，不产出不可靠的百分比。*

四、S2603 / S2604（§ 5.2 热点覆盖，70% 门槛）

分母见《LX5000 卷》第四部分 三张 Top-20 表。该分母口径未决（= 本卷第二部分 2.2）——按字面执行分母含 pthread_mutex_lock、read、sendmsg、vDSO 等选手无法向量化的项（S2604 自有代码仅 8/20，即使全部向量化也永远达不到 70%），故本节只出事实，不给覆盖率。

4.1 各提交的向量化方向（客观清单）

PR	向量化内容	含 RVV 的源文件
S2603 #2	CRC64、HyperLogLog、位运算 (and/andnot/not)、rvv_memchr (9 处) 与 rvv_memcpy (5 处)	crc64_rvv.c、hyperloglog.c、rvv.c/h
S2603 #3	memchr/memcmp/memcpy/memmove 的 RVV 版本	rvv.c/h
S2603 #4	RESP 扫描、内存操作、bitops、HLL、LZF、HNSW	rvv_resp.c、rvv_mem.c、bitops.c、hyperloglog.c、lzf_c.c、hnsw_rvv.c
S2603 #6	仅 rvv_optim.c	rvv_optim.c
S2604 #1	rvv.c/h, hash.c 中 1 处接入	rvv.c/h
S2604 #2	rvv.c	rvv.c
S2604 #3	MurmurHash3 就地向量化 (53 处 intrinsic)	murmur3_hash.c、rvv_optim.c

4.2 已人工核实的热点命中（证据确凿）

PR	热点函数	证据
S2603 #2	processInputBuffer 的核心路径	src/networking.c:3069/3225/3304 三处调用 rvv_memchr(c->querybuf+c->qb_pos, ...)——正是 RESP 内联／多块解析的换行符扫描
S2603 #4	processInputBuffer 的核心路径	src/networking.c:74/79 在 respScanFindByte() 内调用 redisRespScanFindByteRvv; 另有选手自带的运行时覆盖 计数器 src/rvv_coverage.c, 其记录的 13 条路径含 resp_scan、bitcount、lzf_match、hll_dense_* 等。另 有性能实证：09-20 补测显示其 pipeline=64 场景对基 线 +19.1%、pipeline=16 为 +12.1%，而六个非流水线 场景仅 +0.5% 至 +4.4%——增益集中于协议解析占比最 高的场景，与本项命中相互印证
S2604 #3	MurmurHash3_x86_32 (Top-20 第 18 位, 1.44%)	murmur3_hash.c 就地含 53 处 __riscv_v* 调用

4.3 已核实为「未接入」的情形

PR	现象	证据
S2603 #3	其 rvv_memchr／rvv_memcmp／rvv_memcpy／rvv_memmove 仅在 rvv.c 内部被调用，未接入任何热点文件	grep -rl 在 src/*.c 中只命中 rvv.c 自身
	另注：其包装函数带长度阈值 $n < MC_RVV_MEM_THRESHOLD ? memchr(...) : rvv_memchr(...)$ (src/rvv.h:50-52)，即短数据仍走标量	—
S2603 #6	热点相关文件中 RVV 调用数为 0	—
S2604 #1 / #2	8 个自有代码热点中，无一在其定义文件内含 intrinsic 或 RVV 调用	—

4.4 尚待人工逐函数核实的部分

组委会的自动判据是文件级的：只要 networking.c 含 RVV 调用，该文件定义的全部热点函数都会被计入。这会过度归因——processInputBuffer 确实调用了向量化扫描（已核实），但同文件的 writeToClient、addReply、handleClientsWithPendingWrites 等是否真的走到向量化路径，必须逐函数看调用链。

文件级统计结果（仅供排序参考，不可作为覆盖率）：

	base 场景（自有 16 项）	pipe16 场景（自有 14 项）
S2603 #2	文件级 10 项（其中 1 项已人工核实）	文件级 5 项
S2603 #4	文件级 7 项（其中 1 项已人工核实）	文件级 3 项
S2603 #3 / #6	0	0

S2604 的 8 个自有代码热点中，仅 #3 的 MurmurHash3_x86_32 一项经核实为就地向量化（1/8）；#1、#2 为 0/8。

S2603 #4 自带 `src/rvv_coverage.c` 运行时计数器，按 13 条命名路径 × 6 个长度分桶统计 `calls_rvv` vs `calls_scalar`——这是 § 5.2 「或其调用的核心路径被向量化」最强的实证手段，可在压测中直接观测 RVV 路径是否真被走到，建议采信为该提交的补充证据。

五、启动其余三题前须解决的口径问题

下表的编号对应 本卷第二部分 的小节号，便于与该文件对照。

#	问题	影响范围	状态
1	S2603/S2604 分母含不可向量化项（= 本卷第二部分 2.2，内部编号 D10）	7 个 PR	未决
2	P2601 #1 的并行库是否计入 130 条分母（= 2.5）	该 PR 全部判定	未决
3	「默认构建启用」对「默认关、文档开」的提交如何认定（= 2.6）	S2603 #6（ <code>BUILD_RVV=yes</code> 默认关）、S2604 #2（ <code>--enable-riscv-rvv</code> 默认关）	未决，已列入
4	S2605 #1 的 <code>crc32c</code> ：存在 RVV 路径但在验证平台崩溃，是否记为覆盖（= 2.7）	1 个模块	未决，已列入
5	文档矛盾： <code>ruy</code> 单列为独立评分项， <code>ACCEPTANCE_MEMO.md</code> :35 写成已定， <code>REMAINING_WORK.md</code> :73 写「待确认」，而所引条目正文未提 <code>ruy</code>	S2602	须澄清

已于 09-20 解除的阻塞：S2602 分母由「约 105」定稿为 98 项（门槛 89/98），计数单位改为（文件，函数）对，并剔除 `legacy_optimized_ops.h` 的 9 项。据此 S2602 的判定规模为 6 个提交 × 98 项 = 588 条（此前按旧分母估作 630 条，作废）。依据见本文第三章，已登记为已决事项。

六、方法学说明

- 不可裸 `grep __riscv_vector`：各提交的守卫宏不统一——S2602 #1/#2/#3 用 `USE_RVV`、S2603 #2 用 `HAVE_RISCV_RVV`、#6 用 `REDIS_RVV`、S2604 #2 用 `configure` 开关 `--enable-riscv-rvv`、P2601 #2/#3/#4 用 `KLEIDICV_ENABLE_RVV`。裸 `grep` 会漏判。
- 「落在 `rvv` 命名空间」≠「函数体真的向量化」：P2601 #4 把 `baseline` 命名空间整体宏替换为 `rvv`（`kleidicv/include/kleidicv/config.h.in`:68），名义上整批命中，内部是否真向量化须逐个抽查。该题的判定须辅以人工抽查，本表将标注「待人工抽查」。
- 净改动提取须用 `git merge-base origin/HEAD HEAD`，不可硬编码首个 `commit`——提交仓库是上游完整 `fork`，曾把 55 个真实改动算成 8022 个（《LX5000 卷》第一部分 § 5）。
- 可选强证据：对板上已编出的产物按 `TU` 统计 `objdump -d | grep -c vsetvl`，验证「真的编进去了」而非只是源码里存在。

第四部分 验证平台环境定版快照

日期：2026-09-22 编制：组委会验证侧 采集方式：只读脚本，未改动板上任何状态
适用范围：本次复现验证所用的三台机器。本文件为定版基准，此后如平台发生变更须另行记录并说明影响范围。
版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

结论先行：三台机器的操作系统、内核、工具链与 ISA 通告均已定版，记录于第一章。其中三项通告事实直接支撑既有报告结论（第二章）；另有三项环境约束将影响尚未完成的工作，须在排期时一并考虑（第三章）。21 个上板工作树的冻结 SHA 与本地归档、与报告记录三方一致（第四章）。

一、三台机器的定版参数

项	蓝芯 LX5000 (1x)	A210 b1	A210 b2
承担赛题	S2603 / S2604 / S2605	P2601	S2602
发行版	Ubuntu 24.04.4 LTS	Debian 13 (trixie)	Debian 13 (trixie)
内核	6.18.3+	6.6.0	6.6.0
架构	riscv64	riscv64	riscv64
核心数	32	8	8
内存	251 GiB	7.6 GiB	7.6 GiB
VLEN (实测 vlenb×8)	128	128	128
mvendorid	0x489	0x5b7	0x5b7
marchid	0x8000000000000109	0x80000000090c0d00 / 0x8000000009140d00	同 b1
默认 gcc	13.3.0	14.2.0	14.2.0
gcc-13 / g++-13	13.3.0 / 13.3.0	未安装	13.3.0 / 13.3.0
gcc-14 / g++-14	14.2.0 / 14.2.0	14.2.0 / 14.2.0	14.2.0 / 14.2.0
cmake	3.28.3	3.31.6	3.31.6
ninja	未安装	1.12.1	未安装
qemu-riscv64	未安装	10.0.13	10.0.13 (09-20 补装)
binutils	2.42	2.44	2.44
libc6	2.39	2.41	2.41
python3	3.12.3	3.13.5	3.13.5
工作区余量	248 GiB	16 GiB	8.7 GiB (/work 已用 7.2 GiB)

采集时刻：2026-09-20T10:05-10:07Z。b1 与 b2 主机名均为 zhihe，靠 ssh 别名区分。

1.1 ISA 通告全文

LX5000:

rv64imafdcvh_zicbom_zicbop_zicboz_ziccrse_zicntr_zicond_zicsr_zifencei_zihintntl_zihintpause_zihpm_zimop_zaamo_zalrsc_zawrs_zfa_zfbfmin_zfh_zca_zcb_zcd_zcmop_zba_zbb_zbs_zkr_zkt_zvbb_zvbc_zve32f_zve32x_zve64d_zve64f_zve64x_zvfbfmin_zvfbfwmaz_vvfz_vvkb_zvkg_zvkned_zvknhb_zvksed_zvksh_zvkt_smaia_smmmp_smmnm_smmststateen_ssaiasscofpmf_ssnpm_sstc_svade_svinval_svpapot_svpbmt

A210 b1: rv64imafdcv_sscfpmf_svpbmt

A210 b2: rv64imafdcv_zicntr_zicsr_zifencei_zihpm_zba_zbb_zbs_sscfpmf_svpbmt

b1 与 b2 的通告不同但硬件一致（mvendorid/marchid/mimpid 三者相同），系设备树通告差异，已在《A210 卷》第三部分 第 2.1 节查证并记录。

二、直接支撑既有报告结论的三项通告事实

序	事实	支撑的结论
1	LX5000 的 ISA 通告不含 zbc （含向量的 zvvb/zvbc，但无标量 Zbc）	S2605 #1 以 -march=rv64gcv_zbc_zvbc 构建，DB:::0open 即因标量 clmul 非法指令崩溃。见《LX5000 卷》第一部分 第 3.3 节
2	LX5000 的 ISA 通告含 zicbop	S2603 #4 以 -march=rv64gcv（不含 zicbop）构建，编译期宏 __riscv_zicbop 未定义致 Prefetch 被整体禁用，而平台本身支持该扩展。见 本卷第一部分 第 2.3 节
3	三台机器的实测 VLEN 均为 128	VLEN 256 与 512 两档无硬件可用，只能经 QEMU 用户态模拟（-cpu rv64,v=true,vlen=N）。P2601 的三档验证即以此法在 b1 完成

三、影响未完成工作的三项环境约束

序	约束	影响
1	b2 未安装 <code>qemu-riscv64</code> 已于 09-20 解除	已装 qemu-user 10.0.13 并实测三档可用（请求 vlen=128/256/512，程序读 vlenb CSR 回报一致）。S2602 算子级精度的三档执行不再受阻
2	b1 未安装 <code>gcc-13/g++-13</code>	若须在 b1 复现任何依赖 gcc-13 的构建（如 S2602 的 #4 与 #7 所用的 3 参数定点 intrinsics），当前不可行。P2601 四个提交均以 gcc-14 构建，暂不受影响
3	b2 工作区余量 8.7 GiB，而单个 LiteRT 构建树约 0.8 GiB	S2602 六个提交如需并存多个构建树，余量偏紧。已确立「不删构建树」的操作纪律（删除会连带毁掉已抓取的依赖，触发重新抓取失败），两者存在冲突，扩容或清理须在算子级精度开跑前解决

补充：LX5000 与 b2 均未安装 `ninja`，现有构建全部经 Make，本身不构成障碍，仅记录在案。

四、冻结 SHA 的三方核验

21 个上板工作树（S2604 #4 已于 09-01 判无效，未上板）的 `git rev-parse HEAD` 与本地 `pr-archive/` 归档、与 本卷第二部分 第五章附录的记录三方逐一相符，无一例外。

机器	工作树位置	数量
LX5000	<code>~/rvspoc-verify/src/rvspoc-S260*</code>	11（S2603 四个、S2604 三个、S2605 四个）
A210 b1	<code>/work/*.FROZEN_SHA</code>	4（P2601 四个）
A210 b2	<code>/work/s2602/src/*.FROZEN_SHA</code>	6（S2602 六个）

同日另已核验：评委材料中全部 34 条永久链接（第五章附录的 22 条冻结版本链接，加正文引用的 12 条代码文件链接）经逐条请求均返回 HTTP 200，5 个仓库均为 public 可匿名访问；其中 10 条带行号的深链，其行号所指内容与引文逐字相符。

五、采集方法

采集脚本为只读，不写入板上任何路径（除 /tmp 下的一个临时 VLEN 探测程序）。VLEN 取自 vlenb CSR (csrr %0, 0xc22) 乘 8，非从 ISA 字符串推断。ISA 通告取自 /proc/cpuinfo 的 isa 行，即内核依设备树所报，**不等于硬件实际实现的扩展集**——b1 与 b2 的差异即为此类。凡结论依赖某扩展是否可用者，均另以实际执行（构建并运行）验证，不单凭通告字符串。

第五部分 各题 Neon 算子分母表

版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

内部评审工作文档，不进 Git、不对外。生成日期：2026-08-31。用于「已实现 ARM Neon 优化的算子中至少 90% 需提供 RVV 版本」的评审口径。源码版本与赛题一致：KleidiCV 26.03、LiteRT v2.1.4 (含 ruy @ 3286a34c)、RocksDB v11.1.1。

0. 使用说明

- 评审时逐项核对：选手 PR 中该算子是否存在 __riscv_vector 路径且在默认构建中启用 → 记「覆盖」
- 覆盖率 = 覆盖条目数 / 分母条目数，≥ 90% 判「过」；只判过/不过，不参与细分排名
- 本表口径为「分发入口 / 模块级函数」，与备忘录 § 5.1 一致

1. P2601 — KleidiCV 26.03

计数单位：**C API 入口函数 (kleidicv.h 暴露的 API)**。共 130 个，分布于 44 个算子族。凡 _api.cpp 分发到 kleidicv::neon:: 实现的入口计入分母 (canny 为 Neon-only，直接赋函数指针，计入)。

类别	算子族	API 数
analysis	min_max	7
analysis	standalone_lucas_kanade_alg	1
analysis	sum	1
analysis	canny (Neon-only, 无 api 文件)	1
arithmetics	absdiff	5
arithmetics	add	8
arithmetics	add_abs_with_threshold	1
arithmetics	bitwise_and	1
arithmetics	compare (equal/greater)	2
arithmetics	exp	1
arithmetics	in_range	2
arithmetics	multiply	5
arithmetics	scale	2
arithmetics	sub	8

arithmetics	threshold	1
conversions	float_conv	4
conversions	gray_to_rgb	2
conversions	merge	1
conversions	rgb_to_rgb	8
conversions	rgb_to_yuv	6
conversions	split	1
conversions	yuv_to_rgb	5
filters	blur_and_downsample	2
filters	gaussian_blur	3
filters	median_blur	16
filters	scharr	2
filters	separable_filter_2d	4
filters	sobel	4
morphology	dilate/erode	2
resize	resize_linear	14
transform	remap	6
transform	rotate	1
transform	transpose	1
transform	warp_perspective	2

90% 门槛 = 117/130 个 API。评审对照表（含全部 130 个 API 名称）见附录 A。

2. S2602 — LiteRT v2.1.4

计数单位：含 Neon 实现的模块级函数/内核（`tfLite/kernels/internal/optimized/` 下），共 4 个来源：

#	模块	Neon 函数数	说明
1	<code>optimized_ops.h</code> (float & int8 激活/elementwise/pool/reduction 等)	35	含模板实例 (int8/uint8/float)
2	<code>neon_tensor_utils.cc</code>	29	Neon* 系列工具函数（矩阵-向量乘、量化、LSTM cell 组件等）
3	<code>integer_ops/</code> (add/mul/mean/pooling/depthwise_conv/lut)	14	见明细
4	depthwiseconv 系列 (float/uint8/3x3/transitional)	16	见明细
5	<code>legacy_optimized_ops.h</code>	10	GEMVForLstmCell、ShuffledFullyConnected 等

6	resize_bilinear.h	5	
	合计（去重后）	约 105	

计入分母的 **ruy**（GEMM 库 @ 3286a34c）Neon 内核：Kernel8bitNeon*（4 个变体）、KernelFloat32Neon*（4 个变体）、以及 pack_arm 的 Neon 路径。ruy 的 Kernel 总分母 = 6 个 ARM 内核 + x86 内核（avx/avx2/avx512，不属于 Neon 分母）。**ruy 建议作为独立评分项**：选手若用 ruy 自带 RVV 支持或自行移植，单独核验，不与上表合并计百分比（理由：ruy 是外部依赖而非 LiteRT 内核，规则原文针对「LiteRT 中的推理内核」）。

90% 门槛 = 约 95/105 个函数。明细见附录 B。

3. S2605 — RocksDB v11.1.1

结论先行：RocksDB 原生 Neon 实现极少，分母 = 3 个模块，90% 实际等于全覆盖。

#	模块	文件	Neon 内容
1	crc32c_arm64	util/crc32c_arm64.h/.cc	CRC32C 的 ARMv8 crypto 指令实现 + PMULL 变体 (crc32c_pmul_runtime_check); 含手工预取
2	XXPH3	util/xxph3.h	vendored xxHash, NEON 路径 (XXPH_VECTOR == XXPH_NEON)
3	XXH32/64	util/xxhash.h	同上, vendored
—	bloom filter	util/bloom_impl.h / util/dynamic_bloom.h	无 Neon 实现 (BloomMath 等均为标量)
—	memtable 插入、序列化	—	无 Neon 实现

90% 门槛 = 3/3。评审时重点反而是赛题必做项列表里的「布隆过滤器位图查找、SST 序列化/反序列化、CRC32C」——这三项是赛题要求向量化，与 Neon 分母表无关（bloom/序列化本就无 Neon 版本）。

附录 A：P2601 全部 130 个 API 明细（按算子族）

再生成命令（在 KleidiCV 26.03 原版源码根目录执行）：

```
for f in $(grep -rl "neon" kleidicv/src --include="*_api.cpp"); do
  grep -oE "kleidicv_[a-z0-9_]+_(s8|u8|s16|u16|s32|u32|s64|u64|f32)\b" "$f"
done | sort -u
# 上式输出 119 个；另加 canny (grep "kleidicv_canny_u8" kleidicv/src/analysis/
canny_neon.cpp)
# 及无 dtype 后缀的 6 个：kleidicv_bitwise_and / _merge / _split / _rotate / _transpose /
_rgba_to_bgra_u8 已含，合计 130

### bitwise_and (1)
kleidicv_bitwise_and

### merge (1)
kleidicv_merge

### split (1)
kleidicv_split
```

```

### rotate (1)
kleidicv_rotate

### transpose (1)
kleidicv_transpose

### blur_and_downsample (2)
kleidicv_blur_and_downsample_stripe_u8
kleidicv_blur_and_downsample_u8

### canny (1)
kleidicv_canny_u8

### compare_equal (1)
kleidicv_compare_equal_u8

### compare_greater (1)
kleidicv_compare_greater_u8

### dilate (1)
kleidicv_dilate_u8

### erode (1)
kleidicv_erode_u8

### exp (1)
kleidicv_exp_f32

### f32_to (2)
kleidicv_f32_to_s8
kleidicv_f32_to_u8

### gaussian_blur (3)
kleidicv_gaussian_blur_arbitrary_stripe_u8      kleidicv_gaussian_blur_fixed_stripe_u8
kleidicv_gaussian_blur_u8

### gray_to (2)
kleidicv_gray_to_rgb_u8
kleidicv_gray_to_rgba_u8

### in_range (2)
kleidicv_in_range_f32
kleidicv_in_range_u8

### median_blur (16)
kleidicv_median_blur_f32
kleidicv_median_blur_large_hist_stripe_u8
kleidicv_median_blur_s16
kleidicv_median_blur_s32      kleidicv_median_blur_s8
kleidicv_median_blur_small_hist_stripe_u8
kleidicv_median_blur_sorting_network_stripe_f32
kleidicv_median_blur_sorting_network_stripe_s16
kleidicv_median_blur_sorting_network_stripe_s32

```

```

kleidicv_median_blur_sorting_network_stripe_s8
kleidicv_median_blur_sorting_network_stripe_u16
kleidicv_median_blur_sorting_network_stripe_u32
kleidicv_median_blur_sorting_network_stripe_u8    kleidicv_median_blur_u16
kleidicv_median_blur_u32
kleidicv_median_blur_u8

### min_max (6)
kleidicv_min_max_f32    kleidicv_min_max_s16
kleidicv_min_max_s32
kleidicv_min_max_s8    kleidicv_min_max_u16
kleidicv_min_max_u8

### min_max_loc (1)
kleidicv_min_max_loc_u8

### remap (6)
kleidicv_remap_f32_u16    kleidicv_remap_f32_u8
kleidicv_remap_s16_u16
kleidicv_remap_s16_u8    kleidicv_remap_s16point5_u16
kleidicv_remap_s16point5_u8

### resize (13)
kleidicv_resize_1ch_r2_stripe_u8    kleidicv_resize_1ch_r3_stripe_u8
kleidicv_resize_2ch_r2_stripe_u8
kleidicv_resize_2ch_r3_stripe_u8    kleidicv_resize_2x2_stripe_u8
kleidicv_resize_3ch_r2_stripe_u8
kleidicv_resize_3ch_r3_stripe_u8    kleidicv_resize_4x4_stripe_u8
kleidicv_resize_generic_stripe_u8
kleidicv_resize_linear_f32    kleidicv_resize_linear_stripe_f32
kleidicv_resize_linear_u8
kleidicv_resize_to_quarter_u8

### rgb_to (10)
kleidicv_rgb_to_bgr_u8    kleidicv_rgb_to_bgra_u8
kleidicv_rgb_to_rgb_u8
kleidicv_rgb_to_rgba_u8    kleidicv_rgb_to_yuv420p_stripe_u8
kleidicv_rgb_to_yuv420sp_stripe_u8
kleidicv_rgb_to_yuv422_u8    kleidicv_rgb_to_yuv444_u8
kleidicv_rgb_to_yuv_semiplanar_u8
kleidicv_rgb_to_yuv_u8

### rgba_to (4)
kleidicv_rgba_to_bgr_u8    kleidicv_rgba_to_bgra_u8
kleidicv_rgba_to_rgb_u8
kleidicv_rgba_to_rgba_u8

### s8_to (1)
kleidicv_s8_to_f32

### saturating_absdiff (5)
kleidicv_saturating_absdiff_s16    kleidicv_saturating_absdiff_s32
kleidicv_saturating_absdiff_s8
kleidicv_saturating_absdiff_u16

```

```

kleidicv_saturating_absdiff_u8

### saturating_add (8)
kleidicv_saturating_add_s16          kleidicv_saturating_add_s32
kleidicv_saturating_add_s64
kleidicv_saturating_add_s8          kleidicv_saturating_add_u16
kleidicv_saturating_add_u32
kleidicv_saturating_add_u64
kleidicv_saturating_add_u8

### saturating_add_abs_with_threshold (1)
kleidicv_saturating_add_abs_with_threshold_s16

### saturating_multiply (5)
kleidicv_saturating_multiply_s16      kleidicv_saturating_multiply_s32
kleidicv_saturating_multiply_s8
kleidicv_saturating_multiply_u16
kleidicv_saturating_multiply_u8

### saturating_sub (8)
kleidicv_saturating_sub_s16          kleidicv_saturating_sub_s32
kleidicv_saturating_sub_s64
kleidicv_saturating_sub_s8          kleidicv_saturating_sub_u16
kleidicv_saturating_sub_u32
kleidicv_saturating_sub_u64
kleidicv_saturating_sub_u8

### scale (2)
kleidicv_scale_f32
kleidicv_scale_u8

### scharr (2)
kleidicv_scharr_interleaved_s16_u8
kleidicv_scharr_interleaved_stripe_s16_u8

### separable_filter_2d (4)
kleidicv_separable_filter_2d_stripe_u16      kleidicv_separable_filter_2d_stripe_u8
kleidicv_separable_filter_2d_u16
kleidicv_separable_filter_2d_u8

### sobel (4)
kleidicv_sobel_3x3_horizontal_s16_u8
kleidicv_sobel_3x3_horizontal_stripe_s16_u8
kleidicv_sobel_3x3_vertical_s16_u8
kleidicv_sobel_3x3_vertical_stripe_s16_u8

### standalone_lucas_kanade_alg (1)
kleidicv_standalone_lucas_kanade_alg_u8

### sum (1)
kleidicv_sum_f32

### thread_resize_linear_u8 (1)
kleidicv_thread_resize_linear_u8

```

```

### threshold (1)
kleidicv_threshold_binary_u8

### u8_to (1)
kleidicv_u8_to_f32

### warp_perspective (2)
kleidicv_warp_perspective_stripe_u8
kleidicv_warp_perspective_u8

### yuv420p (1)
kleidicv_yuv420p_to_rgb_stripe_u8

### yuv422 (1)
kleidicv_yuv422_to_rgb_u8

### yuv444 (1)
kleidicv_yuv444_to_rgb_u8

### yuv_semiplanar (1)
kleidicv_yuv_semiplanar_to_rgb_u8

### yuv_to (1)
kleidicv_yuv_to_rgb_u8

```

附录 B: S2602 函数明细 (按文件)

```

``` optimized_ops.h (35): AddElementwise, AddScalarBroadcast, AffineQuantize,
ArgMaxVector, ArgMinVector, AveragePool,
CalculateSignedClampingWithRangeBitMasks,
CalculateUnsignedClampingWithRangeBitMasks, ClampWithRangeAndStore,
Dequantize, FindMaxValue, HardSwish, Logistic, LstmCell, MaxPool,
MaximumElementwise, MaximumScalarBroadcast, MinimumElementwise,
MinimumScalarBroadcast, MulElementwise, MulSimpleBroadcast, PReluElementWise,
PReluScalarBroadcast, Quantize, RoundToNearest, RoundToNearestUnsigned,
SaturateAndStore, SaturatingRounding, ShuffledFullyConnected,
ShuffledFullyConnectedWorkerImpl, SoftmaxInt8LUT, StoreValue, Tanh,
aarch64_lookup_vector

neon_tensor_utils.cc (29): AccumulateNeonLane, MulAdd,
MultiplyByQuantizedMultiplier2Rows, NeonApplyLayerNorm, NeonApplySigmoid,
NeonApplyTanhImpl, NeonAsymmetricQuantizeFloats, NeonCwiseAdd,
NeonCwiseClipping, NeonCwiseMul, NeonIsZeroVector,
NeonMatrixBatchVectorAccumulateImpl, NeonMatrixBatchVectorMultiplyAccumulate,
NeonMatrixBatchVectorMultiplyAccumulateImpl, NeonMatrixBatchVectorMultiplyImpl,
NeonMatrixScalarMultiplyAccumulate, NeonMeanStddevNormalization, NeonMinMax,
NeonReductionSumVector, NeonSparseMatrixBatchVectorMultiplyAccumulate,
NeonSparseMatrixBatchVectorMultiplyAccumulate1x16,
NeonSparseMatrixBatchVectorMultiplyAccumulate1x4, NeonSub1Vector,

```

NeonSymmetricQuantizeFloats, NeonVectorBatchVectorCwiseProductAccumulate, NeonVectorScalarMultiply, NeonVectorVectorDotProduct, RoundToNearest

integer\_ops/ (14): AddElementwiseInt16, AddElementwiseInt8, AddScalarBroadcast (add.h) MulElementwise, MulSimpleBroadcast (mul.h) MeanImpl (mean.h, reduce.h) AveragePool, MaxPool (pooling.h) DepthwiseConvInitAccBuffer, Run (depthwise\_conv.h) DoDepthwiseConvHybridGeneral (depthwise\_conv\_hybrid.h) LookupTable (lut.h)

depthwiseconv 系列 (16): Run, RunMult, vdotq\_four\_lane\_s32, vdotq\_s32, vqtbl4q\_s8 (depthwiseconv\_3x3\_filter\_common.h) DepthwiseConvImpl, Run (depthwiseconv\_float.h) DepthwiseConvGeneral, DepthwiseConvInitAccBuffer, Run (depthwiseconv\_uint8.h) PackMacroBlockNeon, Run, util\_vld1\_x8, util\_vld1q\_x8 (depthwiseconv\_uint8\_3x3 Run 等) KernelMacroBlockIntrinsics, PackMacroBlockIntrinsics, ProcessPerDepthIntrinsics, util\_vst1\_u8, util\_vst1\_x8 (depthwiseconv\_uint8\_transitional.h)

legacy\_optimized\_ops.h (10): GEMVForLstmCell, GEMVForLstmCellWithSymmetricRange, LegacyFullyConnectedAsGEMVWorkerImpl, LegacyInt8FullyConnectedAsGEMVWorkerImpl, Logistic, LstmCell, ShuffledFullyConnected, Softmax, Tanh

resize\_bilinear.h (5): Load8IntoLowerS16, Load8IntoUpperU16, Move8IntoUpperU16, ResizeBilinearKernel, ResizeBilinearKernel2x2